

Blind ECDSA from the ECDSA Assumption

Jules Maire¹ and Alan Pulval-Dady²

¹ DIENS, École normale supérieure, PSL University, CNRS, INRIA, Paris, France

² Research Institute CODE, Universität der Bundeswehr, München, Germany

Abstract. Blind signatures have become a cornerstone for privacy-sensitive applications such as digital cash, anonymous credentials, and electronic voting. The elliptic curve variant of the Digital Signature Algorithm (ECDSA) is widely adopted due to its efficiency in resource-constrained environments, such as mobile devices and blockchain systems. Building blind ECDSA is hence a natural goal. One presents the first such construction relying solely on the ECDSA assumption. Despite the inherent complexities in integrating blindness with ECDSA, we design a protocol that ensures both unforgeability and blindness without introducing new computational assumptions and ensuring concurrent security. It involves zero-knowledge proofs based on the MPC-in-the-head paradigm for complex statements combining relations on encrypted elliptic curve points, their coordinates, and discrete logarithms.

Keywords: blind signatures · ECDSA · MPC-in-the-head

1 Introduction

Blind signatures, introduced by Chaum in 1982 [Cha82], are a foundational cryptographic primitive designed to provide both authenticity and privacy. They enable a signer to produce a digital signature on a message without learning its content, achieved through a “blinding process” performed by the requester prior to signing. Once unblinded, the signature remains verifiable as originating from the signer while preserving the confidentiality of the signed message. This capability has established blind signatures as a critical part in privacy-sensitive applications such as digital cash, anonymous credential systems, and electronic voting.

The *Elliptic Curve Digital Signature Algorithm* (ECDSA) is a widely used elliptic curve variant of the *Digital Signature Algorithm*. ECDSA leverages the properties of elliptic curves over finite fields to generate compact signatures that are computationally efficient to create and verify. These properties have made it a standard choice in various resource-constrained environments.

Let G be a generator of a predefined elliptic curve group of prime order q , and let $d \in \{0, \dots, q-1\}$ and $P = dG$ the private and public keys, respectively. To sign a message m , the signer first computes the hash value $z = H(m)$, where H is a cryptographic hash function. A random nonce $k \in \{0, \dots, q-1\}$ is then selected, and the point $Q = kG$ is computed. The x -coordinate of Q , denoted r , is reduced modulo q to form the first component of the signature. The second component s is computed as

$$s = k^{-1}(z + rd) \mod q, \quad (1)$$

where k^{-1} is the modular inverse of k modulo q , the signature is the pair (r, s) .

ECDSA’s security primarily relies on two hard problems: the *Elliptic Curve Discrete Logarithm Problem* that underpins the secrecy of the private key and the *Collision Resistance* of the underlying hash function H . The provable security of ECDSA is a nuanced

E-mail: jules.maire@ens.fr (Jules Maire), alan.pulval-dady@unibw.de (Alan Pulval-Dady)



topic (see e.g. [Bro05, FKP16, GS22, HK23]) as it does not have a security reduction to a well-understood problem. The security of ECDSA depends critically on the unpredictability of the nonce k used in signing. If k is reused or partially predictable, an attacker can recover the private key d using simple algebraic techniques. Even though ECDSA does not have a formal security proof, this scheme has withstood extensive cryptanalysis for many years, earning widespread trust in practical applications. Consequently, its security (namely, unforgeability against chosen-message attacks) is considered a standard cryptographic assumption, supported by decades of scrutiny from the cryptographic community.

Challenges. Designing a blind ECDSA scheme is inherently challenging due to the structural dependencies in the ECDSA signing process and its reliance on the relationship between the private key, message hash, and random nonce. The main difficulties stem from the following factors:

- A. *Dependency on the hash value of the message z* : to achieve blindness, the signature requester has to obscure $z = H(m)$ in some way, making it non-trivial to design a security reduction to the sole ECDSA assumption (without having access to the message m itself).
- B. *Dependency on the nonce k* : in ECDSA, the signature components are directly tied to the random ephemeral value k , which must remain secret from the signature requester (for unforgeability) but also from the signer (for blindness).
- C. *Nonlinearity of the signing equation (1)*: ECDSA relies on modular arithmetic with inverses, which is inherently nonlinear. Incorporating a blinding factor into the signing equation often disrupts this delicate structure, and naive approaches (e.g., blinding z by adding a random offset) alter the relationship between z , k and d .

Having a blind ECDSA protocol that produces exact ECDSA signatures is crucial in practice. It ensures compatibility with existing systems and applications that already rely on ECDSA signatures, avoiding the need for modifications or additional infrastructure.

Prior works. Yi and Lam proposed the first blind ECDSA protocol in 2019 [YL19]. To achieve blindness, the protocol leverages a variant of the Paillier cryptosystem with homomorphic properties modulo q , where q is the elliptic curve group order. However, we demonstrate in Appendix B that the homomorphic cryptosystem fails to achieve semantic security, meaning the scheme cannot ensure blindness. Additionally, the authors provide no formal security analysis for the unforgeability property of their scheme, leaving its overall security uncertain even if the insecure homomorphic encryption scheme were replaced with a secure alternative.

In 2021, Qin, Cai, and Yue [QCY21] introduced another blind ECDSA protocol, also relying on a homomorphic encryption scheme modulo q . They suggested instantiating their protocol with either the (insecure) encryption scheme from [YL19] or the additive homomorphic Castagnos–Laguillaumie (CL) encryption scheme [CL15]. The unforgeability of their protocol is analyzed in a novel, highly specific framework they introduced, called the *Algebraic Bijective Random Oracle Model*, which blends ideas from the Bijective Random Oracle Model [FKP16] and the Algebraic Group Model [PV05, FKL18]. While the security arguments within this model are plausible, relying on such strong and unconventional assumptions is not ideal. It is generally preferable to design a protocol whose security relies solely on the well-established ECDSA assumption, ensuring broader acceptance and trustworthiness.

Recently, two works leveraging ECDSA have emerged. The first one, by Frigo and Shelat [Fs24], introduces a novel anonymous credential system built on ECDSA. Their scheme enables privacy-preserving authentication via zero-knowledge proofs that demonstrate possession of a valid ECDSA signature on an attribute without revealing the signature or the attribute itself. While targeting different applications, their zero-knowledge proofs of knowledge of a SHA preimage are concurrent with our approach.

The second work, by Woo, Ozdemir, Sharp, Pornin, and Grubbs [WOS⁺25], proposes proof-of-possession mechanisms for various *pre-quantum* signature schemes, including one for ECDSA. Their goal is to enable users to prove ownership of a private signing key without revealing it, typically in key registration settings. To this end, they develop succinct zero-knowledge proofs of knowledge for the ECDSA signing relation, incorporating both SHA-related commitments and range proofs. While their trust model assumes a registration authority and differs from our adversarial setting, the techniques they use for proving knowledge of a SHA preimage or of the signer’s randomness are particularly relevant to our work.

Contributions of the paper. We propose a two-round blind signature protocol compatible with ECDSA, i.e., producing standard ECDSA signatures, whose security relies solely on the ECDSA assumption. To achieve such security, the one-more unforgeability reduction must, in some way, extract the message chosen by the signature requester from the blinded value $z = H(m)$. To address this challenge and the associated difficulty **(A)**, we rely on the MPC-in-the-head paradigm [IKOS07, IKOS09]. This technique has recently found applications in pre-quantum cryptography, enabling zero-knowledge arguments of knowledge for discrete logarithms [Jou23], RSA plaintexts, double discrete logarithms [MV23], and factorization of Blum integers [MV25].

To address the second difficulty **(B)**, it becomes necessary to use a two-party secret sharing of k . Given the challenges associated with difficulty **(C)**, a natural and practical approach is to use a multiplicative 2-out-of-2 sharing, setting $k = k_u \cdot k_s \bmod q$, where k_u is chosen by the requester and k_s by the signer. This design is reminiscent of the strategy adopted in [FW24], where the requester first commits to their randomness, preventing them from biasing their randomness based on the signer’s later choices. As in [YL19, QCY21], the computation in (1) is performed using a linearly homomorphic encryption scheme modulo q . This enables shared operations on the signature components, which allows blinding of the message and nonce while ensuring compatibility with the ECDSA signature format.

Along the way, we also propose new zero-knowledge arguments combining statements in the elliptic curve group and in finite fields of different prime order. In particular, we significantly improve the state-of-the-art for proving the knowledge of a committed discrete logarithm of an elliptic curve point, i.e., proving the knowledge of a scalar ω and a point S such that $\omega R = S$ where R is public, ω and the coordinates of S are committed. To place our contributions in context, we revisit prior approaches for proving knowledge of committed discrete logarithms. Faz-Hernandez, Ladd, and Maram [FLM22] designed ZKAttest, a zero-knowledge proof of knowledge of a valid ECDSA signature under a committed public key. Celi, Levin, and Rowell [CLR24] showed that ZKAttest suffers from soundness issues that invalidate its security proof, and they provided a practical attack on the public implementation of ZKAttest. To address this, they introduced two protocols, CDLSS and CDLSD, along with a correct security analysis. More recently, Boneh, Haitner, and Lindell [BHLS25] used Bulletproofs [BBB⁺18] to present compact proofs of knowledge of a committed discrete logarithm, albeit with higher computational complexity than [CLR24]. We therefore build on the approach of Celi *et al.* [CLR24], with one of our contributions being an improvement in communication complexity. Their proofs (as well as those from [FLM22]) are Schnorr-like proofs [Sch91] that rely on binary challenges.

Consequently, a cheating prover has probability $1/2$ of convincing a verifier, so the protocol must be repeated $O(\lambda)$ times to achieve negligible soundness error. Our combination of Schnorr-like proofs with the MPC-in-the-head paradigm is designed to overcome this repetition barrier.

Roadmap. Our main contribution, a generic blind ECDSA construction, is presented in Section 4 and relies on several non-interactive zero-knowledge protocols (NIZK). In particular, we require a zero-knowledge argument (ZKA) of knowledge for a committed discrete logarithm. One builds such a ZKA, denoted **ComDLog**, which internally invokes a subprotocol **AddPoint** for proving knowledge of committed curve points A, B, C satisfying $A + B = C$. Both **AddPoint** and **ComDLog** respectively outperform protocol CDLSS and CDLSD [CLR24]. Hence, we begin by presenting them as an initial (and independent) contribution in Section 3. The non-interactive version of **ComDLog** is introduced in Section 5, together with specific variants for the languages arising in our construction. Section 6 presents two additional NIZK constructions. Finally, Section 7 describes the instantiation, performances, and comparisons.

2 Preliminaries

Notation. Let λ denote the security parameter. We write $\mathbb{F}_p$ for a finite field with p elements, where p is a prime power, and $\mathbb{Z}_m$ for the ring of integers modulo m . We denote by $\llbracket a \rrbracket$ a secret sharing of a , by $[N]$ the set $\{1, \dots, N\}$, and by $[a, b]$ the set of integers $\{a, \dots, b\}$. The notation $a \xleftarrow{\$} \mathcal{S}$ indicates that a is sampled uniformly from a set $\mathcal{S}$. We use PPT to refer to probabilistic polynomial-time algorithms, and $(x_P, y_P) = P$ for the coordinates of an elliptic-curve point P . Finally, given a cryptographic scheme **Algo** consisting of several algorithms, we write **Algo.A** to denote a specific algorithm **A** within it.

In Appendix A, we present a formal definition of digital signature schemes and their security properties, along with basic definitions related to elliptic curve cryptography.

Elliptic Curve Digital Signature Algorithm (ECDSA). ECDSA is a digital signature scheme based on elliptic curve cryptography, consisting of the following four algorithms: **Setup**, **KeyGen**, **Sign**, and **Verify**.

- **Setup:** Generate a prime number p , an elliptic curve E defined over $\mathbb{F}_p$ of prime order q of 2λ bits and a generator G of E . Output $\mathbf{pp} = (p, E, q, G, \mathcal{H})$ where $\mathcal{H}$ is a cryptographic hash function that produces 2λ bits digests.
- **KeyGen:** Generate the private key $\mathbf{sk} \in \mathbb{Z}_q$ as a random integer and compute the public key $\mathbf{pk} = \mathbf{sk}G$.
- **Sign:** Given a message m and the private key $\mathbf{sk}$:
 - Compute $e = \mathcal{H}(m)$.
 - Select a random integer $k \in \mathbb{Z}_q$, and compute $Q = kG$.
 - Set $r = x_Q \bmod q$. If $r = 0$, repeat.
 - Compute $s = k^{-1}(e + r \cdot \mathbf{sk}) \bmod q$. If $s = 0$, repeat.

Output the signature $\sigma = (r, s)$.

- **Verify:** Given a message m , a signature $\sigma = (r, s)$, and the key $\mathbf{pk}$:
 - Compute $e = \mathcal{H}(m)$.
 - Check if $r, s \in [1, q - 1]$. If not, reject.

- Compute $w = s^{-1} \bmod q$, $u_1 = e \cdot w \bmod q$, and $u_2 = r \cdot w \bmod q$.
- Compute $Q = u_1 G + u_2 \cdot \text{pk}$, and check if $r \equiv x_Q \bmod q$. If true, output 1; otherwise, output 0.

As mentioned in the introduction, ECDSA does not have a security reduction to a well-understood problem (see e.g. [Bro05, FKP16, GS22, HK23]). However, it has withstood extensive cryptanalysis and is considered a standard cryptographic assumption (for elliptic curves where the discrete logarithm is intractable and collision-resistant hash functions). In the following, the ECDSA assumption is the assumption that ECDSA is (t, q_S, ϵ) -unforgeable under chosen message attacks for all polynomials $t, q_S = \lambda^{O(1)}$ and all noticeable functions $\epsilon = \lambda^{-O(1)}$. See Appendix A for the definition.

Blind signatures. A *blind signature scheme* (BS) [Cha82] is a 5-tuple of PPT algorithms (Setup, KeyGen, User, Signer, Verify) where (User, Signer) is an interactive protocol between a user holding a message m and a signer holding a signing key such that the user gets a signature σ of m from the signer without disclosing any information on m . Our ECDSA blind signature is a two-round interactive protocol, and we present a standalone definition for such a protocol:

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$. On input λ , outputs the public parameters pp .
- $\text{KeyGen}(\text{pp}) \rightarrow (\text{sk}, \text{pk})$. On input pp , outputs a key pair (sk, pk) .
- $\text{User}_0(m, \text{pk}) \rightarrow (\text{msg}_{U,0}, \text{st}_{U,0})$. On input a message m and pk , outputs a blinded message $\text{msg}_{U,0}$ and some state information $\text{st}_{U,0}$.
- $\text{Signer}_1(\text{msg}_{U,0}, \text{sk}) \rightarrow (\text{msg}_{S,1}, \text{st}_S)$. On input $\text{msg}_{U,0}$ and sk , outputs a blinded partial signature $\text{msg}_{S,1}$ and some state information st_S .
- $\text{User}_1(\text{msg}_{S,1}, \text{st}_{U,0}) \rightarrow (\text{msg}_{U,1}, \text{st}_{U,1})$. On input $\text{msg}_{S,1}$ and $\text{st}_{U,0}$, outputs a blinded message $\text{msg}_{U,1}$ and some state information $\text{st}_{U,1}$.
- $\text{Signer}_2(\text{msg}_{U,1}, \text{st}_S) \rightarrow (\text{msg}_{S,2}, b)$. On input $\text{msg}_{U,1}$ and st_S , outputs a partial signature $\text{msg}_{S,2}$.
- $\text{User}_2(\text{st}_{U,1}, \text{msg}_{S,2}) \rightarrow \sigma$. On input $\text{msg}_{S,2}$ and $\text{st}_{U,1}$, outputs a bit string σ .
- $\text{Verify}(m, \sigma, \text{pk}) \rightarrow b \in \{0, 1\}$. On input m, σ and pk , outputs a bit b . If $b = 1$, then σ is called a *signature* of m for pk .

Moreover, it satisfies the following correctness property: we have for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Verify}(m, \sigma, \text{pk}) = 1 \mid \begin{array}{l} \text{pp} \xleftarrow{\$} \text{Setup}(1^\lambda), m \xleftarrow{\$} \mathcal{M}, \\ (\text{sk}, \text{pk}) \xleftarrow{\$} \text{KeyGen}(\text{pp}) \\ (\text{msg}_{U,0}, \text{st}_{U,0}) \xleftarrow{\$} \text{User}_0(m, \text{pk}) \\ (\text{msg}_{S,1}, \text{st}_S) \xleftarrow{\$} \text{Signer}_1(\text{msg}_{U,0}, \text{sk}) \\ (\text{msg}_{U,1}, \text{st}_{U,1}) \xleftarrow{\$} \text{User}_1(\text{msg}_{S,1}, \text{st}_{U,0}) \\ (\text{msg}_{S,2}, b) \xleftarrow{\$} \text{Signer}_2(\text{msg}_{U,1}, \text{st}_S) \\ \sigma \xleftarrow{\$} \text{User}_2(\text{st}_{U,1}, \text{msg}_{S,2}) \end{array} \right] = 1.$$

The security of a blind signature scheme, as defined in [PS00], relies on two properties: *one-more unforgeability* and *blindness*.

Definition 1. Let $t, q_S : \mathbb{N} \rightarrow \mathbb{N}$ and $\epsilon : \mathbb{N} \rightarrow [0, 1]$. A blind signature scheme (Setup, KeyGen, User, Signer, Verify) is said (t, q_S, ϵ) -one-more unforgeable if for all algorithms A , we have for all $\lambda \in \mathbb{N}$:

$$\Pr \left[\begin{array}{l} \text{Verify}(m_i, \sigma_i, \text{pk}) = 1 \\ \text{with } m_i \neq m_j \\ \forall i \neq j \in \{1, \dots, \ell\} \end{array} \mid \begin{array}{l} \text{pp} \xleftarrow{\$} \text{Setup}(1^\lambda), \\ (\text{sk}, \text{pk}) \xleftarrow{\$} \text{KeyGen}, \\ \{(m_i, \sigma_i)\}_{i \in [\ell-1]} \xleftarrow{\$} A^{\text{Signer}(\cdot, \text{sk})}(\text{pk}) \end{array} \right] \leq \frac{1}{2} + \epsilon(\lambda)$$

when A runs in time at most $t(\lambda)$ and makes at most $(\ell - 1) \leq q_S(\lambda)$ (concurrent) queries to its signing oracle in this probabilistic computational game.

Definition 2. Let $t, q_S : \mathbb{N} \rightarrow \mathbb{N}$ and $\epsilon : \mathbb{N} \rightarrow [0, 1]$. A blind signature scheme ($\text{Setup}, \text{KeyGen}, \text{User}, \text{Signer}, \text{Verify}$) is said (t, q_S, ϵ) -blind if for all algorithms A , we have for all $\lambda \in \mathbb{N}$:

$$\Pr \left[b^* = b \mid \begin{array}{l} \text{pp} \xleftarrow{\$} \text{Setup}(1^\lambda), (\text{sk}, \text{pk}) \xleftarrow{\$} \text{KeyGen}, \\ (m_0, m_1, s_1) \xleftarrow{\$} A^{\text{User}(\cdot, \text{pk})}(\text{sk}), b \xleftarrow{\$} \{0, 1\} \\ (\text{msg}_{U,0}, \text{st}_{U,0}) \xleftarrow{\$} \text{User}_0(m_b, \text{pk}) \\ (\text{msg}_{S,1}, \text{st}_S, s_2) \xleftarrow{\$} A^{\text{User}(\cdot, \text{pk})}(s_1, \text{msg}_{U,0}) \\ (\text{msg}_{U,1}, \text{st}_{U,1}) \xleftarrow{\$} \text{User}_1(\text{msg}_{S,1}, \text{st}_{U,0}) \\ (\text{msg}_{S,2}, s_3) \xleftarrow{\$} A^{\text{User}(\cdot, \text{pk})}(s_2, \text{msg}_{U,1}, \text{st}_S) \\ \sigma \xleftarrow{\$} \text{User}_2(\text{st}_{U,1}, \text{msg}_{S,2}) \\ b^* \xleftarrow{\$} A^{\text{User}(\cdot, \text{pk})}(s_3, \sigma) \end{array} \right] \leq \epsilon(\lambda)$$

when A runs in time at most $t(\lambda)$ and makes at most $q_S(\lambda)$ queries to its user oracle in this probabilistic computational game.

Zero-knowledge arguments of knowledge. A zero-knowledge argument (ZKA) protocol [GMR89] for some polynomial-time decidable binary relation $\mathcal{R}$ is defined by two PPT interactive algorithms, a prover P and a verifier V : both V and P are given a common input x and P is given in addition a witness w such that $(x, w) \in \mathcal{R}$. Then, P and V exchange a sequence of messages alternatively until V outputs a bit b (with $b = 1$ indicating that V accepts P 's claim and $b = 0$ otherwise). The entire sequence of messages exchanged, along with the answer b , is called the transcript of the protocol denoted $\text{View}(P(x, w), V(x))$.

A ZKA of knowledge for $\mathcal{R}$ with soundness error ϵ , completeness error α and (t, ζ) -zero-knowledge satisfies the following properties:

- α -completeness: For all $\lambda \in \mathbb{N}$ and $(x, w) \in \mathcal{R}$ with $x \in \{0, 1\}^\lambda$, V accepts with probability α .
- ϵ -knowledge soundness: For all $\lambda \in \mathbb{N}$, $x \in \{0, 1\}^\lambda$, and any PPT P^* that convinces V with probability $\epsilon + \nu$ (where ν is some noticeable function), there exists a PPT extractor E such that, with rewindable access to P^* , E outputs w such that $(x, w) \in \mathcal{R}$ with probability at least $1/2$.
- (t, ζ) -zero-knowledge: For all $\lambda \in \mathbb{N}$ and every PPT V^* , there exists a PPT simulator S such that, given $x \in \{0, 1\}^\lambda$, $\text{View}(P(x, w), V^*(x))$ is (t, ζ) -indistinguishable from the output of S .

If the zero-knowledge property holds only for the genuine verifier V , then the protocol is deemed *honest-verifier* zero-knowledge.

Non-interactive zero-knowledge arguments (NIZK). These public-coin interactive ZKA can be transformed into non-interactive protocols in the random oracle model via generic transforms (e.g., Fiat-Shamir [FS87] or Fischlin [Fis05]). One denotes such NIZK proof system by $\text{NIZK}_{\mathcal{L}} = (\text{NIZK.Setup}, \text{NIZK.P}, \text{NIZK.V})$ for a language $\mathcal{L}$, and one may use the notation $\text{NIZK.P}(x; w; \mathcal{L})$ (and $\text{NIZK.V}(x; w; \mathcal{L})$) when $(x, w) \in \mathcal{R}(\mathcal{L})$. One focuses on NIZK achieving straight-line extractability (SLE) where the extractor algorithm E can output a witness w such that $\mathcal{R}(x, w)$ holds without rewinding P^* and just by observing the instance output x and proof π of P^* such that $\text{NIZK.V}(x, \pi) = 1$.

MPC-in-the-head (MPCitH). The MPCitH paradigm [IKOS07, IKOS09] constructs ZKA from secure multi-party computation (MPC) protocols. Let $\mathcal{L} \in \mathcal{NP}$ and $(x, w) \in \mathcal{R}(\mathcal{L})$, then there exists a function f computable in polynomial time such that $f(x, w) = 1$. The MPCitH paradigm considers a prover P and a verifier V engaged in a two-party interactive protocol, where P seeks to convince V that they know a witness w such that $\mathcal{R}(x, w) = 1$ for some public statement x . P typically splits their secret w into N shares $\llbracket x \rrbracket_1, \dots, \llbracket x \rrbracket_N$, and simulates an N -party MPC protocol to compute $f(x, w)$. At the end of the simulation, P commits to the views of the N parties (i.e., each party's input, randomness, and messages) using a hash-based commitment scheme. V then sends a challenge asking P to open a subset of the views. P responds by revealing the requested views, and V checks both the consistency of these views with the simulated MPC protocol and the correctness of the commitment openings. In the basic setting where all-but-one views are opened, the resulting zero-knowledge protocol achieves a soundness error of $1/N$.

We use, without going into too much detail, well-known optimizations for MPCitH. Notably the rooted Goldreich-Goldwasser-Micali pseudorandom function framework [GMR88] to generate $N = 2^t$ pseudorandom λ -bit strings (the leaves) within a structured depth- t binary tree. This design enables revealing $N - 1$ pseudorandom leaf values by disclosing only $\log_2 N$ nodes, specifically the sibling nodes along the path from the root to the unrevealed leaf.

Other cryptographic primitives. Let ℓ be a prime power. We make use of linear homomorphic encryption schemes LHE_ℓ (with algorithms Setup , KeyGen , Enc , Dec), operating over a group $\mathbb{G}_\ell$ of order ℓ . In addition to the hash-based commitments used in our MPCitH proofs, we also consider linearly homomorphic group-based commitment schemes Com_ℓ (with algorithms Setup , Com , Ver), where the message space is $\mathbb{Z}_\ell$ and the commitments are vectors of group elements. For simplicity, we may omit the randomness in the commitment notation. See Appendix A for formal definitions.

3 Zero-Knowledge Argument for Committed Discrete Logarithm: **ComDLog**

Let p, q be prime numbers, and E, E' be two elliptic curves such that $|E(\mathbb{F}_p)| = q$ and $|E'(\mathbb{F})| = p$ for some arbitrary finite field $\mathbb{F}$.

Let us consider two elliptic curve points $S, R = (x, y) \in E(\mathbb{F}_p)$, a discrete logarithm relation $\omega R = S$ among them (with $\omega \in \mathbb{F}_q$), and three commitments $C_1 = \text{Com}_p(x)$, $C_2 = \text{Com}_p(y)$, $C_3 = \text{Com}_q(\omega)$. The goal of this section is to derive a zero-knowledge protocol for this committed discrete logarithm relation, labeled **ComDLog**. In this protocol, a prover $P(R, \omega, \{C_i\}_{i \in [3]})$ aims to convince a verifier $V(R, \{C_i\}_{i \in [3]})$ that they know a point S and a scalar ω such that $\omega R = S$, where the coordinates of S are committed in C_1 and C_2 , and ω is committed in C_3 .

The first step consists of a protocol, **AddPoint**, that proves knowledge of three committed points $A, B, C \in E(\mathbb{F}_p)$ satisfying the relation $A + B = C$. As a preparatory step, we introduce **SubAddPoint**, a zero-knowledge protocol for proving multiplicative relations between committed values.

3.1 Zero-knowledge argument of multiplicative relations between committed values and commitments to zero: **SubAddPoint**

Let us consider the following scenario: given 10 committed scalars $\{x_i, y_i, z_i, x_i y_i, x_i z_i\}_{i \in [2]}$ the prover wants to convince the verifier that they know the committed values (and the corresponding randomness used in the commitments) and that these committed values

satisfy the multiplicative relations. On top of performing this task, **SubAddPoint** aims the prover to prove that they know the randomness used in two commitments to 0 (namely w_1, w_2).

Although this language seems *ad hoc*, it has been designed to fit with the next protocol **AddPoint**.

Protocol 1 - SubAddPoint

Input: For $i \in [2]$, P and V hold:

$$\begin{aligned} X_i &= \text{Com}_p(x_i; r_i), Y_i = \text{Com}_p(y_i; s_i), Z_i = \text{Com}_p(z_i; t_i), \\ U_i &= \text{Com}_p(x_i y_i; u_i), V_i = \text{Com}_p(x_i z_i; v_i), W_i = \text{Com}_p(0; w_i). \end{aligned}$$

Moreover, all these committed values and corresponding randomness are known to P . Additionally, let us fix a public subset $\mathbb{S} \subset \mathbb{F}_p$.

Goal: P wants to prove:

- (i) the values committed in $\{U_i, V_i\}_{i \in [2]}$ are products of values committed in $\{X_i, Y_i, Z_i\}_{i \in [2]}$,
- (ii) $\{W_1, W_2\}$ are commitments to 0.

Round 1: P computes two random commitments $A_1 = \text{Com}_p(\alpha_1; \beta_1)$, $A_2 = \text{Com}_p(\alpha_2; \beta_2)$ and a commitment $A_3 = \text{Com}_p(0; \beta_3)$ to 0, and sends them to V .

Round 2: V sends three batching challenges $\gamma_1, \gamma_2, \gamma_3 \xleftarrow{\mathbb{S}}$. Then, P and V compute for $i \in [2]$:

$$\begin{aligned} B_i &= Y_i + \gamma_i Z_i = \text{Com}_p(y_i + \gamma_i z_i), \\ C_i &= U_i + \gamma_i V_i = \text{Com}_p(x_i(y_i + \gamma_i z_i)), \\ W_3 &= W_1 + \gamma_3 W_2 = \text{Com}_p(0; \omega_1 + \gamma_3 \omega_2). \end{aligned}$$

Round 3: P computes and sends $A_{4,1} = \alpha_1 B_1$, $A_{4,2} = \alpha_2 B_2$ to V .

Round 4: V sends a challenge $c \xleftarrow{\mathbb{S}}$ (*à la* Schnorr) and a fourth batching challenge $\gamma_4 \xleftarrow{\mathbb{S}}$ to P .

Round 5: P computes and sends to V the following values:

$$\begin{aligned} a_i &= \alpha_i + c x_i \bmod p \text{ for } i \in [2], \\ b_i &= \beta_i + c r_i \bmod p \text{ for } i \in [2], \\ a_3 &= \beta_3 + c(w_1 + \gamma_3 w_2) \bmod p, \\ a_4 &= x_1(s_1 + \gamma_1 t_1) + \gamma_4 x_2(s_2 + t_2 \gamma_2) - c(u_1 + \gamma_1 v_1) \\ &\quad - \gamma_4 c(u_2 + \gamma_2 v_2) \bmod p. \end{aligned}$$

Verification: V accepts the proof if and only if the following equations are

satisfied:

$$\begin{aligned} \text{Com}_p(a_1; b_1) &= A_1 + cX_1, \\ \text{Com}_p(a_2; b_2) &= A_2 + cX_2, \\ \text{Com}_p(0; a_3) &= A_3 + cW_3, \\ a_1B_1 + \gamma_4a_2B_2 &= A_{4,1} + A_{4,2}\gamma_4 + (C_1 + C_2\gamma_4)c + \text{Com}_p(0; a_4). \end{aligned}$$

Theorem 1 (SubAddPoint security proofs). *Given the inputs $\{X_i, Y_i, Z_i, U_i, V_i, W_i\}_{i \in [2]}$, SubAddPoint (Protocol 1) is an honest-verifier zero-knowledge argument of knowledge for proving that*

- the values committed in $\{U_i, V_i\}_{i \in [2]}$ are product of values committed in $\{X_i, Y_i, Z_i\}_{i \in [2]}$,
- $\{W_1, W_2\}$ are commitments to 0,

achieving perfect completeness and $\frac{2}{|\mathbb{S}|}$ -knowledge soundness, where $\mathbb{S}$ is a parameter of SubAddPoint.

Proof. See Appendix C. □

3.2 Zero-knowledge argument for a sum of committed points: AddPoint

To prove the knowledge of points over $E(\mathbb{F}_p)$ satisfying the relationship $A + B = C$ whose coordinates have been committed, we employ the standard formula for point addition on elliptic curves described in the Appendix A and rely on SubAddPoint as follows.

Protocol 2 - AddPoint

Input: Given $\{A = (a_x, a_y), B = (b_x, b_y), C = (c_x, c_y)\} \in E(\mathbb{F}_p)$, P and V hold $\text{Com}_p(a_x), \text{Com}_p(a_y), \text{Com}_p(b_x), \text{Com}_p(b_y), \text{Com}_p(c_x)$, and $\text{Com}_p(c_y)$. Moreover, P knows these committed values and corresponding randomness.

Goal: Proving the knowledge of the committed points A, B, C satisfying $A + B = C$.

Step 1: P computes these commitments and sends them to V:

$$\begin{aligned} X_1 &= \text{Com}_p((b_x - a_x)^{-1}), \quad X_2 = \text{Com}_p\left(\frac{b_y - a_y}{b_x - a_x}\right), \\ Y_1 &= \text{Com}_p(b_x - a_x), \quad Y_2 = X_2, \\ Z_1 &= \text{Com}_p(b_y - a_y), \quad Z_2 = \text{Com}_p(a_x - c_x), \\ U_1 &= \text{Com}_p(1), \quad U_2 = \text{Com}\left(\left(\frac{b_y - a_y}{b_x - a_x}\right)^2\right), \\ V_1 &= X_2, \quad V_2 = \text{Com}_p\left(\left(\frac{b_y - a_y}{b_x - a_x}\right)(a_x - c_x)\right), \\ W_1 &= \text{Com}_p(c_x) - (U_2 - \text{Com}_p(a_x) - \text{Com}_p(b_x)), \\ W_2 &= \text{Com}_p(c_y) - (V_2 - \text{Com}_p(a_y)). \end{aligned}$$

Step 2: P and V realize the protocol SubAddPoint on inputs $\{X_i, Y_i, Z_i, U_i, V_i, W_i\}_{i \in [2]}$.

Implementing Com_p with the Pedersen commitment (defined in Appendix A), the communication complexity of our protocol (given that commitments computed in Step 1 are already known to V) is 5 curve points and 6 scalars, whereas the protocol [CLR24] requires the communication of 14 curve points and 24 scalars. This represents an improvement of more than 70% in terms of bits.

3.3 Zero-knowledge argument for a committed discrete logarithm: ComDLog

Recall that we deal with two points $R, S \in E(\mathbb{F}_p)$ such that $S = \omega R := (x, y)$, for some scalar $\omega \in \mathbb{F}_q$. Given commitments $C_1 = \text{Com}_p(x; r_1), C_2 = \text{Com}_p(y; r_2), C_3 = \text{Com}_q(\omega; r_3)$ to x, y, ω , we want to build a ZKA of knowledge of S and ω satisfying the discrete logarithm in basis R .

Sketch of the protocol. This proof relies on the MPCitH framework. As described in the preliminaries (Section 2), P emulates N virtual parties who then perform some MPC protocol. Let us describe it. As inputs, parties receive sharings $\llbracket \omega \rrbracket$ and $\llbracket r_3 \rrbracket$ of w and r_3 . Then each party $i \in [N]$ performs the following computation:

1. They compute $\llbracket \omega \rrbracket_i R := (x_i, y_i) := \llbracket S \rrbracket_i$;
2. They compute $C_{1,i} = \text{Com}_p(x_i; r_{1,i}), C_{2,i} = \text{Com}_p(y_i; r_{2,i}), C_{3,i} = \text{Com}_q(\llbracket \omega \rrbracket_i; \llbracket r_3 \rrbracket_i)$, where $r_{1,i}, r_{2,i}$ are randomly chosen (defining commitments for each party);
3. They broadcast $C_{1,i}, C_{2,i}, C_{3,i}$.

Once these computations are performed, P commits to $\{C_{1,i}, C_{2,i}, C_{3,i}\}_{i \in [N]}$, and opens all-but-one views depending on V's challenge. V recomputes all these $\{C_{1,i}, C_{2,i}, C_{3,i}\}_{i \neq i^*}$ and obtains $C_{1,i^*}, C_{2,i^*}, C_{3,i^*}$ from P to recompute the hash commitment. From these open views, V can check that $\llbracket w \rrbracket_i R = (x_i, y_i) := \llbracket S \rrbracket_i$ for all $i \neq i^*$, and can be convinced that:

1. $\llbracket w \rrbracket$ encodes the value committed in C_3 by checking the equation $C_3 = \sum_i C_{3,i}$;
2. $\llbracket S \rrbracket$ encodes the coordinates committed in C_1, C_2 by checking $(x, y) = \sum_{i \neq i^*} \llbracket S \rrbracket_i + \llbracket S \rrbracket_{i^*}$ using **AddPoint**, since x, y and $\llbracket S \rrbracket_{i^*}$ are committed.

In short, this proves that the values inside these commitments $\{C_{1,i}, C_{2,i}, C_{3,i}\}_{i \in [N]}$ are shares encoding committed values in C_1, C_2, C_3 and that they realize a discrete logarithm in basis R .

For the sake of simplicity we choose $\mathbb{S} = \mathbb{F}_p$ in **AddPoint** for the rest of the section.

Protocol 3 - ComDLog

Input: $R \in E(\mathbb{F}_p), C_1 = \text{Com}_p(x; r_1), C_2 = \text{Com}_p(y; r_2), C_3 = \text{Com}_q(\omega; r_3)$ are public parameters, whereas $\{r_1, r_2\} \in \mathbb{F}_p, \{r_3, \omega\} \in \mathbb{F}_q$ are known to P.

Round 1: P produces commitments of the MPCitH part:

1. Computes a PRG tree with N leaves seed_i , derives randomness, and computes each party's shares $\{\llbracket r_3 \rrbracket, \llbracket \omega \rrbracket\}$ (with auxiliary value $\{\Delta r_3 := r_3 - \sum_{i=1}^N \llbracket r_3 \rrbracket_i, \Delta \omega := \omega - \sum_{i=1}^N \llbracket \omega \rrbracket_i\}$).
2. Derives $\llbracket S \rrbracket = \llbracket \omega \rrbracket R, \Delta S = \Delta \omega R$, and $(x_i, y_i) := \llbracket S \rrbracket_i$ for every $i \in [N]$.
3. Computes commitments $C_{1,i} = \text{Com}_p(x_i; r_{1,i}), C_{2,i} = \text{Com}_p(y_i; r_{2,i})$, and $C_{3,i} = \text{Com}_q(\llbracket \omega \rrbracket_i, \llbracket r_3 \rrbracket_i)$, with $r_{1,i}, r_{2,i}$ from the randomness source of the party, for every $i \in [N]$.
4. Computes $h = \mathcal{H}(\Delta \omega, \Delta r_3, \text{Com}(\text{seed}_i)_{i \in [N]}, \{C_{\ell,i}\}_{\ell \in [3], i \in [N]})$ where seed_i denotes the i -th leaf of the PRG tree.
5. Computes $\{A_1 = \text{Com}_p(\alpha_1, \beta_1), A_2 = \text{Com}_p(\alpha_2, \beta_2), A_3 = \text{Com}_p(0, \beta_3)\}$ (round 1, step 2 of **AddPoint**).

P sends (h, A_1, A_2, A_3) to V.

Round 2: V produces the challenges:

1. $i^* \xleftarrow{\$} [N]$ for the party's views not to open.
2. $\gamma_1, \gamma_2, \gamma_3 \xleftarrow{\$} \mathbb{F}_p$ (round 2 of **AddPoint**).

V_s ends these 4 challenges to P.

Round 3: P opens the parties' views:

1. Given $A = (x, y)$, $B = -\llbracket S \rrbracket_{i^*}$, $C = \sum_{i \neq i^*} \llbracket S \rrbracket_i + \Delta S$, P computes the commitments X_1, X_2, U_1, U_2, V_1 as defined in step 1 of **AddPoint**.
2. The other commitments can be deduced by V since C is going to be revealed. Then, P computes the commitments $A_{4,1}, A_{4,2}$ (round 3 of **AddPoint**), and sends these 7 commitments to V.
3. P opens the views to all the parties except the i^* -th, by sending the co-path in the PRG tree for the seed_{i^*} . Moreover, P sends the commitments of the i^* -th party, and $\{\Delta\omega, \Delta r_3, \{C_{j,i^*}\}_{j \in [3]}\}$.

Round 4: V checks that open views are consistent with the commitments:

1. For each $i \neq i^*$, V derives the seed seed_i , computes the commitment $\text{Com}(\text{seed}_i)$, the shares, the auxiliary values, and the commitments $\{C_{j,i}\}_{j \in [3]}$.
2. Computes and checks h , and possibly halts.
3. Checks $C_3 = \sum_i C_{3,i}$, and possibly halts.
4. Sends challenges $c \xleftarrow{\$} \mathbb{F}_p$ and $\gamma_4 \xleftarrow{\$} \mathbb{F}_p$ (round 3 of **AddPoint**) to P.

Round 5: P computes $a_1, a_2, a_3, a_4, b_1, b_2$ as in round 5 of **AddPoint**, and sends them to V.

Round 6: V realizes the verification step of **AddPoint** and possibly halts.

Remark 1. The communication complexity of **AddPoint** remains unchanged when one of the three points involved in the addition is public and not committed. Moreover, deriving the sharing of S from the one of ω ensures us that $\{\llbracket S \rrbracket_i\}_{i \in [N]}$ are curve points, and thus the addition is well-defined.

Complexity. The communication complexity in bits is

$$2\lambda + \tau(24 \log |\mathbb{F}| + 2 \log q + 8 \log p + \lambda \log N + 2\lambda).$$

To determine the computational complexity, let us take a look into the MPC modeling. The parties compute $\llbracket \omega \rrbracket_i R := (x_i, y_i)$ and $C_{1,i}, C_{2,i}, C_{3,i}$ (plus the auxiliary value $\Delta S = \Delta\omega R$). Hence, the computational complexity of P for the MPCitH part is $3N + 1$ scalar multiplications over $E(\mathbb{F}_p)$, and $4N$ scalar multiplications over $E'(\mathbb{F})$. P also needs to compute: A_1, A_2, A_3 (5 scalar multiplications over $E'(\mathbb{F})$), X_1, X_2, U_1, U_2, V_1 (7 scalar multiplications over $E'(\mathbb{F})$), $A_{4,1}, A_{4,2}$ via B_1, B_2 (4 scalar multiplications over $E'(\mathbb{F})$). The overall computational complexity of P is then $4N + 16$ scalar multiplications over $E'(\mathbb{F})$, and $3N + 1$ scalar multiplications over $E(\mathbb{F}_p)$ per repetition of the protocol. Similarly, the overall computational complexity of V is $4(N - 1) + 15$ scalar multiplications over $E'(\mathbb{F})$, and $3(N - 1) + 1$ scalar multiplications over $E(\mathbb{F}_p)$ per repetition.

Theorem 2 (ComDLog security proofs). *ComDLog protocol 3 is an honest-verifier zero-knowledge argument of knowledge for the relation*

$$\mathcal{R} = \{(x, y, \omega; R, C_1, C_2, C_3) \in (\mathbb{F}_p)^2 \times \mathbb{F}_q \times E(\mathbb{F}_p) \times (E'(\mathbb{F}))^2 \times E(\mathbb{F}_p) \mid$$

$$\text{Com}_p(x) = C_1, \text{Com}_p(y) = C_2, \text{Com}_q(\omega) = C_3, \omega R = (x, y)\},$$

with perfect completeness and special soundness ϵ equals to $\max(\frac{2}{|\mathbb{S}|}, \frac{1}{N})$, where N is the number of parties emulated by the prover, and $\mathbb{S}$ is a parameter of the protocol.

Proof. See Appendix C. □

Hypercubing technique. ComDLog can be compiled with the hypercubing technique from [AGH⁺23], which offers an interesting trade-off between communication and computation complexity. In a nutshell, given $N = k^d$ parties, one can represent their N shares into a hypercube of dimension d and length k . By fixing one dimension, one can regroup the N shares into a disjoint union of k subsets. By summing the shares in the same subset, one gets k new shares such that the overall sum of these k shares equals the sum of the N original shares. Hence, we can apply the MPC protocol on these k shares. This reasoning can be extended to each other's dimension (and it has to be extended to reach a soundness of $1/N$). It leads to d MPC protocols with k parties. Eventually, instead of emulating 1 MPC with N parties (for each repetition), for the same soundness error, P can emulate $\log_2 N$ MPC with 2 parties. This yields to a reduction from N to $2 \log_2 N$ parties to simulate, and this can even be reduced to $\log_2 N + 1$ parties (since after running one MPC, we know the output for the $\log_2 N - 1$ other MPC protocols).

This technique was introduced to reduce the size of proofs by allowing N to be larger because of the lower number of parties to be emulated. Paradoxically, this technique increases ComDLog proof size but remains interesting for applications where the verifier needs to be efficient. Indeed, the drawback of hypercubing our protocol is that one needs to generate one AddPoint proof per MPC. Hence, instead of one AddPoint per repetition of ComDLog, one needs to run d independent AddPoint per repetition (i.e., one for each of the d MPC). The proof size in bits is then

$$2\lambda + \tau(2 \log q + \lambda \log N + 2\lambda + \log N(24 \log |\mathbb{F}| + 8 \log p)).$$

After hypercubing ComDLog, P performs $20 \log N - 4$ scalar multiplications over $E'(\mathbb{F})$ and $3 \log N - 2$ over $E(\mathbb{F}_p)$. V's computation consists of $19 \log N - 8$ scalar multiplications over $E'(\mathbb{F})$ and $3 \log N - 5$ over $E(\mathbb{F}_p)$.

Remark 2. The Threshold-in-the-Head (TCitH) framework [FR23, FR25] does not fit our setting, since there is no straightforward way, from a Shamir secret sharing of a curve point (x, y) , to implicitly derive a sharing of the commitment of each coordinate x and y (i.e. a sharing of C_1 and C_2), except if the threshold is trivial.

3.4 Performances

We compare our protocol AddPoint with CDLSS [CLR24] for a single repetition on a 256-bit curve. Next, considering $N = 2^8$ parties and a security parameter $\lambda = 128$, we evaluate our protocol ComDLog against CDLSD [CLR24]. To achieve a soundness error below $2^{-\lambda}$, our protocol is repeated $\approx \lambda / \log_2 N$ times, instead of $\approx \lambda$ times as in Schnorr-like protocols. The following table summarizes the computational complexity for P and V (in terms of the number of scalar multiplications), as well as the communication complexity (in kB) of the different protocols.

Protocol	P comput. compl.	V comput. compl.	proof size in kB
CDLSS [CLR24]	26	34	1.6
AddPoint	16	15	0.5
CDLSD [CLR24]	4736	3840	178.7
ComDLog	28944	28816	19.5
ComDLog (Hypercube)	2848	2608	132

ComDLog achieves 89% reduction in proof size compared to CDLSD. Additionally, the hypercubing variant outperforms both in computational complexity (by 32% – 40%) and communication complexity (by 26%).

4 A Blind ECDSA from the ECDSA Assumption

We introduce the main result of this work, a generic construction of a blind ECDSA signature with a security reduction to ECDSA.

Protocol description. We adopt the notation for blind signatures (BS) introduced in the preliminaries (Section 2). Let m denote a message to be signed. The **BS.Setup** algorithm generates the system parameters for both the user and the signer. Subsequently, the signer executes **BS.KeyGen** to generate their ECDSA key pair.

► BS.Setup(1^λ) Generate $E(\mathbb{F}_p)$ of prime order q Generate $E'(\mathbb{F})$ of prime order p Pick a generator G of $E(\mathbb{F}_p)$ $pp_1 \leftarrow \text{LHE}_q.\text{Setup}(1^\lambda)$ $pp_2 \leftarrow \text{LHE}_p.\text{Setup}(1^\lambda)$ $pp_3 \leftarrow \text{Com}_p.\text{Setup}(1^\lambda)$ $pp_4 \leftarrow \text{Com}_q.\text{Setup}(1^\lambda)$ $\mathcal{H}$ a cryptographic hash function $pp \leftarrow (\{pp_i\}_{i \in [4]}, E, p, q, G, \mathcal{H})$ Output pp	► BS.KeyGen(pp) $sk_{\text{ECDSA}} \xleftarrow{\\$} \mathbb{F}_q,$ $pk_{\text{ECDSA}} \leftarrow sk_{\text{ECDSA}} G$ Output $(sk_{\text{ECDSA}}, pk_{\text{ECDSA}})$
--	--

The user generates keys for each encryption scheme using **UserSetup** described in Appendix E. Once these preliminary steps are complete, the user interacts with the signer in rounds through the routines **BS.User**₀, **BS.Signer**₁, **BS.User**₁, **BS.Signer**₂, **BS.User**₂ which we formally define below. To ensure its security, our protocol brings into play four NIZKs for the underlying languages. One omits the domains for the sake of clarity.

$$\mathcal{L}_2 := \{E_m \mid \exists (m, r_{E_m}) \text{ s.t. } E_m = \text{LHE}_q.\text{enc}(\mathcal{H}(m); r_{E_m})\}$$

The language $\mathcal{L}_2$ aims to prove the knowledge of m, r_{E_m} .

$$\mathcal{L}_1 := \left\{ (C_3, G, C_1, C_2, E_u) \left| \begin{array}{l} \exists (Q_u, k_u, r_1, r_2, r_3, r_{E_u}) \text{ s.t. } k_u G = Q_u, \\ C_1 = \text{Com}_p(x_{Q_u}; r_1), C_2 = \text{Com}_p(y_{Q_u}; r_2), \\ C_3 = \text{Com}_q(k_u; r_3), E_u = \text{LHE}_p.\text{enc}(Q_u; r_{E_u}) \end{array} \right. \right\}$$

From the knowledge of m , the security reduction can get (from the ECDSA signing oracle) a signature on m associated with a point $Q = kG$ for some nonce k unknown to the reduction. To embed this implicit value k within the protocol, it becomes necessary to devise a mechanism for obtaining k_u , whence the language $\mathcal{L}_1$.

$$\mathcal{L}_3 := \left\{ (C_{x_Q}, C_{y_Q}, C_3, E_{x_Q}, Q_s) \left| \begin{array}{l} \exists (Q, k_u, r_{E_{x_Q}}, r_{C_1}, r_{C_2}, r_3) \text{ s.t.} \\ C_{x_Q} = \text{Com}_p(x_Q; r_{C_1}), \\ C_{y_Q} = \text{Com}_p(y_Q; r_{C_2}), \\ C_3 = \text{Com}_q(k_u; r_3), k_u Q_s = Q, \\ E_{x_Q} = \text{LHE}_q.\text{enc}(x_Q; r_{E_{x_Q}}) \end{array} \right. \right\}$$

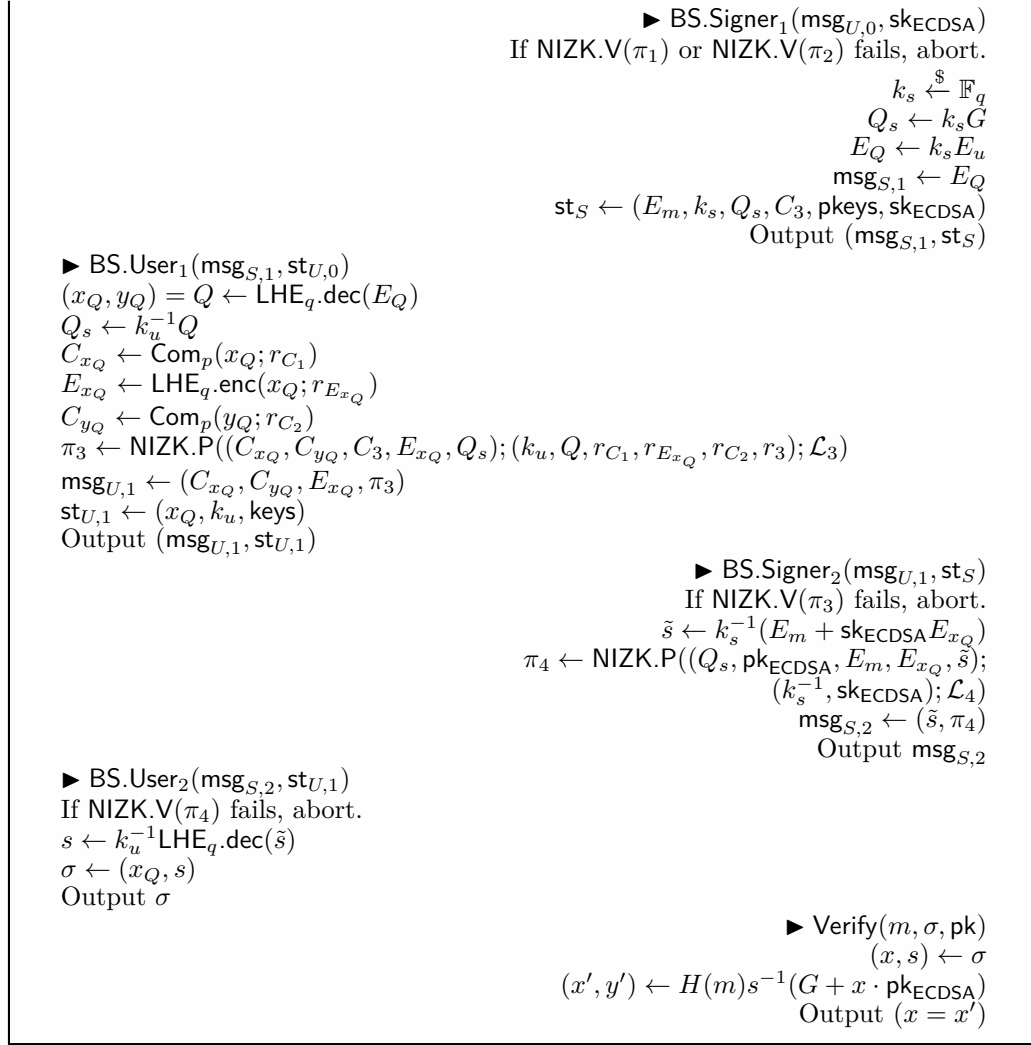
Moreover, since the x -coordinate of Q is crucial for computing the signature as defined in (1), it is imperative to ensure that the signature requester does not cheat and correctly incorporates this abscissa during the protocol execution, whence $\mathcal{L}_3$.

$$\mathcal{L}_4 := \left\{ (E_m, E_{x_Q}, Q_s, \text{pk}_{\text{ECDSA}}, \tilde{s}) \left| \begin{array}{l} \exists (k_s, \text{sk}_{\text{ECDSA}}) \text{ s.t.} \\ \text{pk}_{\text{ECDSA}} = \text{sk}_{\text{ECDSA}} G, Q_s = k_s G, \\ \tilde{s} = k_s^{-1}(E_m + \text{sk}_{\text{ECDSA}} E_{x_Q}) \end{array} \right. \right\}$$

Finally, the language $\mathcal{L}_4$ has been added to provide an additional layer of security on top of the encryption. It aims to prevent the signer from attempting to gain information about the underlying session.

Protocol 4 - Generic blind ECDSA

► BS.User₀($m, \text{pk}_{\text{ECDSA}}, \text{pp}$)
 $(\text{sk}_{\text{LHE}_q}, \text{pk}_{\text{LHE}_q}, \text{sk}_{\text{LHE}_p}, \text{pk}_{\text{LHE}_p}) \leftarrow \text{UserSetup}(\text{pp})$ (see E)
 $k_u \xleftarrow{\$} \mathbb{F}_q$
 $(x_{Q_u}, y_{Q_u}) = Q_u \leftarrow k_u G$
 $E_u \leftarrow \text{LHE}_p.\text{enc}(Q_u; r_{E_u})$
 $E_m \leftarrow \text{LHE}_q.\text{enc}(\mathcal{H}(m); r_{E_m})$
 $C_1 \leftarrow \text{Com}_p(x_{Q_u}; r_1)$
 $C_2 \leftarrow \text{Com}_p(y_{Q_u}; r_2)$
 $C_3 \leftarrow \text{Com}_q(k_u; r_3)$
 $\pi_1 \leftarrow \text{NIZK.P}((C_1, C_2, C_3, E_u, G); (Q_u, k_u, r_1, r_2, r_3, r_{E_u}); \mathcal{L}_1)$
 $\pi_2 \leftarrow \text{NIZK.P}(E_m; (m, r_{E_m}); \mathcal{L}_2)$
 $\text{pkeys} \leftarrow (\text{pk}_{\text{ECDSA}}, \text{pk}_{\text{LHE}_q}, \text{pk}_{\text{LHE}_p})$
 $\text{keys} \leftarrow (\text{pkeys}, \text{sk}_{\text{LHE}_q}, \text{sk}_{\text{LHE}_p})$
 $\text{msg}_{U,0} \leftarrow (\{C_i\}_{i \in [3]}, E_u, E_m, \pi_1, \pi_2, \text{pkeys})$
 $\text{st}_{U,0} \leftarrow (k_u, r_3, C_3, \text{keys})$
 Output ($\text{msg}_{U,0}, \text{st}_{U,0}$)



Theorem 3 (One-More Unforgeability). *Let us consider an algorithm A breaking the one-more unforgeability (OMU) of our blind ECDSA protocol 4. Then there exists an algorithm B breaking the EUF-CMA security of ECDSA with q queries to the signing oracle $\mathcal{O}^{\text{ECDSA.SignKeyGen}}$ or there exists at least one of the two algorithms C_1, C_2 respectively breaking the straight-line extractability of $\text{NIZK}_{\mathcal{L}_1}, \text{NIZK}_{\mathcal{L}_2}$. In particular, for all security parameters λ , we have*

$$\text{Adv}_A(\lambda) \leq q(\text{Adv}_B(\lambda) + \text{Adv}_{C_1}(\lambda) + \text{Adv}_{C_2}(\lambda)) .$$

Proof. Given the existence of an algorithm A breaking the one-more unforgeability of our protocol, one constructs an algorithm B solving the EUF-CMA problem of ECDSA. B first calls the oracle $\mathcal{O}^{\text{ECDSA.KeyGen}}$ that returns a public key pk_{ECDSA} . B sends pk_{ECDSA} to A. Then they repeat the following process q times:

- A sends msg_{U,0} from the output of BS.User₀ (on input pk_{ECDSA} and some message m) to B.
- B tries to extract from π_1 and π_2 the witnesses k_u, r_{E_u}, r_{E_m} and m (thanks to the SLE of our NIZKs). Then, B queries $\mathcal{O}^{\text{ECDSA.SignKeyGen}}$ on message m and gets the ECDSA signature $\sigma = (x, k^{-1}(\mathcal{H}(m) + \text{sk}_{\text{ECDSA}} \cdot x))$.

- If the extraction of π_1 or π_2 fails, then there exists an adversary C_1 or C_2 providing valid but non-extractable proofs, thus breaking the SLE of $\text{NIZK}_{\mathcal{L}_1}$ or $\text{NIZK}_{\mathcal{L}_2}$.
- Otherwise, using the curve equation, B computes a point $Q = (x, y^*)$. Then, using pk_{LHE_p} from $\text{msg}_{U,0}$, B sends $\text{LHE}_p.\text{enc}(Q; r_{E_u})$ as $\text{msg}_{S,1}$.
- A sends $\text{msg}_{U,1}$ from BS.User_1 to B.
- B computes $k_u k^{-1}(\mathcal{H}(m) + \text{sk}_{\text{ECDSA}} \cdot x)$. Then, using pk_{LHE_q} from $\text{msg}_{U,0}$, B computes $\tilde{s} := \text{LHE}_q.\text{enc}(k_u k^{-1}(\mathcal{H}(m) + \text{sk}_{\text{ECDSA}} \cdot x); r_{E_u} + r_{E_m})$ and sends it to A as $\text{msg}_{S,2}$.
- A sends the output of BS.User_2 which is exactly $(x, k^{-1}(\mathcal{H}(m) + \text{sk}_{\text{ECDSA}} \cdot x))$.

After these q iterations, A holds q ECDSA message-signature couples and is able to produce a new couple (m', σ') for some message m' not queried to $\mathcal{O}^{\text{ECDSA.Sig}^\text{n}}$. Finally, A sends it to B who checks that this is indeed a valid signature for a new message. $\square$

Security reduction to ECDSA. The advantage of our construction lies in the extractability of the message m from π_2 . Consequently, the reduction can query the signing oracle $\mathcal{O}^{\text{ECDSA.Sig}^\text{n}}$. By contrast, the concurrent blind ECDSA [QCY21] does not allow direct extraction of m , and therefore must rely on the Algebraic Bijective Random Oracle (ABRO) model, which combines ideas from the Bijective Random Oracle Model [FKP16] and the Algebraic Group Model [PV05, FKL18].

Concurrent security argument. Concurrent security holds for our scheme because the OMU security proof is a reduction to the EUF-CMA security of ECDSA. In particular, the user cannot manipulate the signatures obtained from the signing oracle $\mathcal{O}^{\text{ECDSA.Sig}^\text{n}}$, and therefore cannot exploit them to forge a valid signature.

Theorem 4 (Blindness). *Let us consider an algorithm A breaking the blindness property of the blind ECDSA protocol 4. Then there exists an algorithm B breaking the zero-knowledge of one of the NIZK proofs $\text{NIZK}_{\mathcal{L}_1}$, $\text{NIZK}_{\mathcal{L}_2}$, $\text{NIZK}_{\mathcal{L}_3}$, or there exists an algorithm C breaking the IND-CPA security of one of the encryption schemes LHE_q , LHE_p , or there exists an algorithm D breaking the hiding property of one of the commitment schemes Com_p , Com_q , or there exists an algorithm E breaking the knowledge soundness of $\text{NIZK}_{\mathcal{L}_4}$. In particular, for all security parameters λ , we have*

$$\text{Adv}_A(\lambda) \leq \text{Adv}_B(\lambda) + \text{Adv}_C(\lambda) + \text{Adv}_D(\lambda) + \text{Adv}_E(\lambda) .$$

Proof. Assume the existence of an adversary A breaking the blindness property of Protocol 4. Let us define two oracles: $\mathcal{O}^{\text{User}_0}$, which simulates an honest execution of the user's side, and $\mathcal{O}^{\text{User}_1}$, which is progressively modified to simulate various components of the protocol.

Consider a challenger who chooses a random bit $b \in \{0, 1\}$ and responds to A's queries using $\mathcal{O}^{\text{User}_b}$. The goal of A is to determine the value of b given the transcript.

First, we replace the real NIZK proofs in $\mathcal{O}^{\text{User}_1}$ with simulated ones, using the simulator provided by the zero-knowledge property. If A manages to distinguish them, then this concludes the proof, since it implies the existence of an algorithm B that breaks the zero-knowledge property of one of these proofs, say with advantage $\text{Adv}_B(\lambda)$.

Next, we modify $\mathcal{O}^{\text{User}_1}$ to encrypt a random message instead of the actual one. Since NIZK proofs are simulated, this deviation is not detected. If A manages to distinguish these transcripts, then this concludes the proof, since it implies the existence of an algorithm C that breaks the IND-CPA security of one of the encryption schemes, say with advantage $\text{Adv}_C(\lambda)$.

We then modify $\mathcal{O}^{\text{User}_1}$ to compute random commitments rather than honest ones. This change is undetectable due to the simulated proofs. If A manages to distinguish these

transcripts, then this concludes the proof, since it implies the existence of an algorithm D that breaks the hiding property of one of the commitment schemes, say with advantage $\text{Adv}_D(\lambda)$.

Finally, we modify the oracle $\mathcal{O}^{\text{User}_1}$ such that, upon receiving π_4 , it attempts to extract a witness from π_4 . At this step, since A breaks the blindness property of Protocol 4, there must exist an adversary E breaking the knowledge soundness of $\text{NIZK}_{\mathcal{L}_4}$, say with advantage $\text{Adv}_E(\lambda)$. Indeed, if the extraction succeeds, the oracle will use the extracted secret key sk_{ECDSA} from π_4 to produce a valid signature on the initial message, which would contradict the advantage of A against the blindness property of Protocol 4. Hence, there is no other choice than that the extraction fails, and the existence of E providing valid but non-extractable proofs. $\square$

5 Straight-Line Non-Interactive ComDLog Variants

Section 3 presents ComDLog, a ZKA of knowledge for a committed discrete logarithm that outperforms CDLSS proof [CLR24]. However, ComDLog internally calls AddPoint whose extractor has to rewind 8 times (see proof in Appendix C), preventing us from transforming it into a straight-line NIZK proof.

To address this limitation, we choose to use CDLSS proof that is 2-sound. In a nutshell, CDLSS performs the same task as AddPoint and is similar, with the exception that it does not use batching techniques (whence its straight-line extractability).

By applying the Fischlin transform on the CDLSS part and the Fiat-Shamir transform on the MPCitH part of ComDLog, one consequently obtains a straight-line extractable non-interactive variant of ComDLog.

Fiat-Shamir transform. The Fiat-Shamir transform [FS87] is a technique that converts an interactive public-coin proof into a non-interactive one, in the random oracle model. In the original interactive protocol, P sends a commitment and receives a random challenge from V . The Fiat-Shamir transform replaces V 's random challenge with a deterministic hash of the commitment (and the statement), using a cryptographic hash function modeled as a random oracle. This allows P to compute the challenge on their own, producing a non-interactive proof that can be verified by anyone.

Fischlin's transform. The Fischlin's transform [Fis05] is another technique that converts an interactive public-coin proof into a non-interactive one. As an overview, P first generates τ messages $\vec{m} = (m_1, \dots, m_\tau)$ and then works one proof at a time. For each $i \in [\tau]$, P finds e_i, z_i such that (m_i, e_i, z_i) is an accepting transcript and $\mathcal{H}_b(x, \vec{m}, e_i, z_i) = 0$, where $\mathcal{H}_b$ denotes the output of the random oracle truncated to b bits, x the statement, and e_i, z_i the verifier query and prover response during the i -th iteration. This prevents P from guessing e_i and then going back to find (m_i, z_i) since the entire vector $\vec{m}$ is included in the hash function. Therefore, P must be lucky and guess all $e_1, \dots, e_\tau$ ahead of time. However, the probability of this happening is $2^{-b\tau}$ and so P would need to run expected time $2^{b\tau}$ to cheat. By setting $b\tau \geq \lambda$, it follows that this is infeasible.

5.1 Straight-line non-interactive ComDLog variant

We start by presenting the modifications of ComDLog that yield a straight-line NIZK, denoted NComDLog. Next, we extend NComDLog to the specific languages $\mathcal{L}_1$ and $\mathcal{L}_3$.

Description of NComDLog. As mentioned earlier, for straight line extractibility purposes we focus on a variant of ComDLog in which the internal call to AddPoint is replaced by a call to CDLSS. Non-interactively, P begins by computing τ independent repetitions of the MPCitH part and τ commitments $\{com_j\}_{j \in [\tau]}$ for input material (as in ComDLog). For each of these repetitions, they apply the Fiat-Shamir transform to derive the open view challenge $\{i_j^*\}_{j \in [\tau]}$. Subsequently, P computes τ responses $\{resp_j\}_{j \in [\tau]}$. It leads to τ transcripts $\{t_j := (com_j, i_j^*, resp_j)\}_{j \in [\tau]}$. For each $j \in [\tau]$, P computes the first message m_j of a CDLSS instance (that depends on t_j). Let $\vec{m} := (m_1, \dots, m_\tau)$, then P applies the Fischlin's transform: For each $j \in [\tau]$, P tries to find a challenge c_j (for CDLSS) such that the response z_j satisfies $\mathcal{H}_b(\vec{m}, c_j, z_j) = 0$. The communication cost of this NIZK is then

$$2\lambda + \tau(2\log_2 q + \lambda \log_2 N + 2\lambda + |\pi_{\text{CDLSS}}|) \text{ bits},$$

where π_{CDLSS} is the CDLSS proof. A formal description of NComDLog can be found in Appendix D.

Straight-line extractibility of NComDLog. We show that the probability of a malicious prover holding an invalid witness, successfully cheating in any of the τ repetitions is negligible. First note that if P succeeds in cheating in ρ MPC rounds and κ CDLSS rounds with $\rho + \kappa \geq \tau + 1$, then by the pigeonhole principle, there exists at least one repetition in which P cheats in both rounds, preventing us from extracting a witness in that iteration. Recall that the probability of a successful cheating in the MPC part is $\frac{1}{N}$ and in the CDLSS part is 2^{-b} . Let us denote the events E_1 "the malicious prover succeeds in cheating on ρ MPC" and E_2 "the malicious prover succeeds in cheating on κ CDLSS". One then defines X_1 the random variable following the binomial distribution with parameters $(\tau, \frac{1}{N})$, and X_2 the random variable following the binomial distribution with parameters $(\tau, 2^{-b})$. The probability $\Pr(E_1 \cap E_2)$ that the prover cheats on ρ MPC and κ CDLSS is upper bounded by $\Pr(X_1 \geq \rho) + \Pr(X_2 \geq \kappa)$ which can be written as $\sum_{i=\rho}^{\tau} \binom{\tau}{i} \frac{1}{N^i} (1 - \frac{1}{N})^{\tau-i} + \sum_{i=\kappa}^{\tau} \binom{\tau}{i} 2^{-bi} (1 - 2^{-b})^{\tau-i}$. Hence, by choosing a set of parameters $(N, b, \tau, \rho, \kappa)$ such that $\Pr(E_1 \cap E_2) \leq 2^{-\lambda}$ is negligible, one concludes the sketch of the proof. Essentially, one picks b to be minimum for a given τ and λ (recall that $b\tau \geq \lambda$). Indeed, although b does not affect the amount of exponentiation on the curve, we still want to minimize the number of field operations.

5.2 Zero-knowledge argument for $\mathcal{L}_1$: NIZK $_{\mathcal{L}_1}$

The language $\mathcal{L}_1$ requires proving knowledge of committed values and associated randomness, knowledge of a committed discrete logarithm, and knowledge of an encrypted point with its associated randomness. NComDLog already performs the first two tasks. Let us explain how to deal with proving the knowledge of Q_u, r_{E_u} such that $E_u = \text{LHE}_p.\text{enc}(Q_u; r_{E_u})$. In particular, how to extend NComDLog by incorporating the computation of $\text{LHE}_p.\text{enc}(Q_u; r_{E_u})$ in the MPCitH part.

The coordinates (x_{Q_u}, y_{Q_u}) of the point Q_u are already shared in ComDLog via $\llbracket k_u \rrbracket G = ((x_{Q_{u,i}}, y_{Q_{u,i}}))_{i \in [N]}$. Hence, P has to additionally share the randomness r_{E_u} such that each party i can compute and broadcast $\llbracket E_u \rrbracket_i := \text{LHE}_p.\text{enc}((x_{Q_{u,i}}, y_{Q_{u,i}}); \llbracket r_{E_u} \rrbracket_i)$. Then, E_u is recovered as

$$E_u = \sum_{i \in [N]} \text{LHE}_p.\text{enc}((x_{Q_{u,i}}, y_{Q_{u,i}}); \llbracket r_{E_u} \rrbracket_i) + \text{LHE}_p.\text{enc}(\Delta k_u G; \Delta r_{E_u}).$$

This NComDLog extension introduces minimal complexity overhead. The additional communication cost comes from the sharing of r_{E_u} , i.e., sending Δr_{E_u} . The prover must also commit to each party's computation of $\llbracket E_u \rrbracket$, which is added to the input of the hash

function already used in **ComDLog** to commit to the parties' computations. As a remark, the extractor of **ComDLog** (see proof in Appendix C) can be used to extract r_{E_u} (still with two transcripts), and the soundness error of the proof remains unchanged compared to that of **ComDLog**.

5.3 Zero-knowledge argument for $\mathcal{L}_3$: **NIZK** $_{\mathcal{L}_3}$

At first glance, the language $\mathcal{L}_3$ seems to require the same adaptation as for $\mathcal{L}_1$ since one has to prove the knowledge of committed values and associated randomness, committed discrete logarithm, and encrypted element with its associated randomness. However, a secret value x_Q is shared both modulo p and q during the proof. Indeed, x_Q appears in $C_{x_Q} = \text{Com}_p(x_Q; r_{C_1})$ and in $E_{x_Q} = \text{LHE}_q.\text{enc}(x_Q; r_{E_{x_Q}})$. Therefore, one should prevent a malicious prover from sharing two different values. For this purpose one applies a strategy that leverages a third sharing, namely a sharing over the integers, for proving the equality between encoded values over different moduli.

Sharing over the integers. In [FMRV22], the authors developed a technique for the MPCitH paradigm to encode a secret $x \in \mathbb{Z}_q$ over the integers. In this framework, an element x is shared as $x = \sum_{i=1}^N \llbracket x \rrbracket_i + \Delta x$ where $\llbracket x \rrbracket_i \xleftarrow{\$} [0, A-1]$, with no modular reduction. It was first introduced to share small elements $x < B \ll q$ over large rings $\mathbb{Z}_q$ by taking $B < A \ll q$. This drastically reduces the cost of the sharing for large q . To avoid information leakage, the authors introduced the possibility for P to abort depending on the views to open requested by V . This induces a rejection rate of $\frac{B-1}{A}$ in the protocol that can be decreased by increasing A (at a cost of increasing the communication complexity). This technique can be extended to share any element $x \in \mathbb{Z}_q$ over the integers by tacking $A > q$, and the rejection rate becomes $\frac{q-1}{A}$. We make use of this last extension in the present subsection.

Our strategy. Let us denote as $\llbracket \cdot \rrbracket^{(p)}$ (resp. $\llbracket \cdot \rrbracket^{(q)}$) the sharing modulo p (resp. modulo q). Given two sharings $\llbracket x_Q \rrbracket^{(p)}$ and $\llbracket x_Q \rrbracket^{(q)}$, the Chinese remainder theorem asserts that, if p and q are coprime, then there exists a unique integer $x_Q \leq pq$ such that $x_Q \bmod p = \sum_i \llbracket x_Q \rrbracket_i^{(p)}$ and $x_Q \bmod q = \sum_i \llbracket x_Q \rrbracket_i^{(q)}$. By picking $NA \approx pq$, one can share x_Q over the integers as $x_Q = \sum_{i=1}^N \llbracket x_Q \rrbracket_i + \Delta x_Q$ where $\llbracket x_Q \rrbracket_i \xleftarrow{\$} [0, A-1]$. Then P produces the following hash-based commitments:

$$\text{Com}(\llbracket x_Q \rrbracket_i^{(p)} - \llbracket x_Q \rrbracket_i \bmod p), \text{Com}(\llbracket x_Q \rrbracket_i^{(q)} - \llbracket x_Q \rrbracket_i \bmod q)$$

for all $i \in [N]$. By opening all-but-one party (let us say the i^* -th party), V can recompute all these commitments for $i \neq i^*$ and check the commitments for i^* by computing $\sum_{i \neq i^*} (-\llbracket x_Q \rrbracket_i^{(p)} + \llbracket x_Q \rrbracket_i) \bmod p$ and $\sum_{i \neq i^*} (-\llbracket x_Q \rrbracket_i^{(q)} + \llbracket x_Q \rrbracket_i) \bmod q$. Then V is convinced that the same value is shared modulo p and q during the proof for $\mathcal{L}_3$. This strategy introduces an additional communication cost coming from the sharing over the integers of x_Q . This is the cost of sending $x_Q - \llbracket x_Q \rrbracket_{i^*}$ (which is $\log_2(A-1)$ bits) because V can recover Δx_Q by computing $\Delta x_Q = x_Q - \llbracket x_Q \rrbracket_{i^*} - \sum_{i \neq i^*} \llbracket x_Q \rrbracket_i$. Then, the proof size is

$$2\lambda + \tau(2 \log_2 q + \log_2 p + 2\lambda + \lambda \log_2 N + \log_2(A-1) + |\pi_{\text{CDLSS}}|) \text{ bits},$$

where π_{CDLSS} is the CDLSS proof. The extractor of **ComDLog** can again be used here to extract r_E (still with two transcripts), and the soundness error of the proof remains unchanged even with the sharing over the integer. In contrast, the simulator has to be slightly adapted to integrate the rejection rate as in [FMRV22].

6 Zero-Knowledge Argument for $\mathcal{L}_2$ and $\mathcal{L}_4$

6.1 Zero-knowledge argument for $\mathcal{L}_2$

The language $\mathcal{L}_2$ aims at proving knowledge of a preimage m such that $\text{LHE}_q.\text{enc}(\mathcal{H}(m))$ holds. Representing $\mathcal{H}$ as a Boolean circuit, several constructions provide short proofs for this language [AHIV23, DOT21]. In our work, we adopt the framework of Katz et al. [KKW18] (KKW), which is better suited to our context. KKW offers efficient proofs of knowledge of preimages for large Boolean circuits. It takes as input a Boolean circuit representation C_f of a function f , with a public input x and a secret input w (represented via a Boolean sharing), and outputs the bit $C_f(x, w) = 1 \iff f(x, w) = 1$.

The protocol relies on a preprocessing phase in which the parties receive sharings of correlated randomness. Concretely, they obtain Boolean sharings $\llbracket a \rrbracket, \llbracket b \rrbracket, \llbracket c \rrbracket$ where the Boolean values a, b, c satisfy $a \cdot b = c$, with $\cdot$ denoting the AND gate. To achieve this, the authors introduce a cut-and-choose technique in the MPCitH framework, making the preprocessing verifiable. This approach turns out to integrate elegantly into our setting.

Our strategy. Using KKW, we obtain a straight-line extractable NIZK proof for knowledge of a preimage m given $\mathcal{H}(m)$. However, in the KKW framework, $\mathcal{H}(m)$ is evaluated under a Boolean sharing, whereas $\text{LHE}_q.\text{enc}$ is naturally represented as an arithmetic circuit. Thus, given a Boolean sharing of $\mathcal{H}(m)$, the central question becomes how to convert it into an arithmetic sharing, ideally within the KKW protocol itself. Once such an arithmetic sharing of $\mathcal{H}(m)$ is available, computing

$$\llbracket \text{LHE}_q.\text{enc}(\mathcal{H}(m)) \rrbracket = \text{LHE}_q.\text{enc}(\llbracket \mathcal{H}(m) \rrbracket) \bmod q$$

is straightforward.

Boolean to arithmetic conversion. Such a conversion is in fact a well-studied problem in the masking literature (e.g., [CGV14]), and can naturally leverage the preprocessing phase, making it particularly suitable for the KKW framework. In particular, [HKL21] extended KKW by introducing conversion gates with Boolean input wires and arithmetic output wires.

As in traditional conversion protocols, the preprocessing begins with P pseudorandomly generating, from seeds, correlated randomness $\llbracket b_1 \rrbracket, \llbracket b_2 \rrbracket^B$ such that $b_1 = b_2 \in \mathbb{Z}_2^k$ (where $\llbracket \cdot \rrbracket^B$ denotes Boolean sharing). That is, a random binary string is shared both arithmetically and Booleanly. Here, equality between a Boolean and an arithmetic value simply means both encode the same bit in their respective representations.

For a single-bit conversion, the preprocessing requires $\log q$ bits of communication, since one party receives a non-pseudorandomly chosen value. In the online phase, an additional $\log q$ bits are needed, because P must broadcast the unopened party's message to V along with the views of all opened parties.

Finally, [HKL21] extend this method to convert an entire Boolean vector into a single arithmetic value by interpreting the Boolean vector as the binary representation of that value. This optimization reduces the naïve online translation cost from $\log^2 q$ bits to $\log q$ bits, so that converting a $\log q$ -bit vector requires $\log q(\log q + 1)$ bits in total.

Complexity analysis. To analyze the proof size, we recall the details of KKW, and in particular its use of the cut-and-choose technique. Informally, P commits to a collection of shares that should satisfy a given property. By revealing all but a subset of these shares, V checks that the opened shares are consistent with the property, and is thus convinced (with some probability) that the unopened shares also satisfy it. These unopened shares can then be used by P in the emulated MPC.

More concretely, P generates M pairs of sharing and commits to them. V requests the opening of all but one pair, and verifies that each opened pair encodes an identical value. As a result, V can trust the unopened pair with a soundness error of $1/M$. To reduce the soundness error to negligible, the protocol is typically repeated τ times. As an optimization, rather than opening $M - 1$ pairs per repetition, one can instead keep unopened a subset of $M - \tau$ pairs without repeating the full protocol.

Let $|C|$ denote the circuit size (i.e., the number of AND gates) of $\mathcal{H}$, and let $w = m$ be the input to $\mathcal{H}$. The resulting proof size, based on the analysis of [KKW18], is:

$$2\lambda + \tau \log \frac{M}{\tau} 3\lambda + \tau (\log q (\log q + 1) + \lambda \log N + 2|C| + |m| + 3\lambda) \text{ bits.}$$

Finally, we emphasize that the straight-line extractability of our NIZK ultimately relies on the extractability guarantees of the underlying KKW protocol.

6.2 Zero-knowledge argument for $\mathcal{L}_4$

The language $\mathcal{L}_4$ ensures that P (the signer, in this case) has honestly computed

$$\tilde{s} = k_s^{-1} (E_m + \text{sk}_{\text{ECDSA}} \cdot E_{x_Q}),$$

thereby preventing the signer from gaining information about the session. In essence, P must prove that they used both sk_{ECDSA} and k_s in the computation of $\tilde{s}$, where the ciphertexts E_m, E_{x_Q} of $\mathcal{H}(m), x_Q$ are known to V .

One possible approach, leveraging the MPCitH paradigm, is to share k_s^{-1} and sk_{ECDSA} . Then, only a single multiplication needs to be proven, which can be handled using the usual triplet sacrificing technique.

Concretely, P first samples a sharing of $a \xleftarrow{\$} \mathbb{F}_q$ (as a uniformly random element, no auxiliary value is required). Next, P computes a random sharing $\llbracket c \rrbracket$ along with Δc such that $c = a \cdot \text{sk}_{\text{ECDSA}}$. Then, given the sharing of $k_s^{-1}, \text{sk}_{\text{ECDSA}}, a$, and c as inputs to the parties, P executes the following MPC protocol:

1. The parties get a random challenge $\epsilon \in \mathbb{F}_q$ from V ;
2. They locally set $\llbracket \alpha \rrbracket = \epsilon \llbracket k_s^{-1} \rrbracket + \llbracket a \rrbracket$;
3. They broadcast $\llbracket \alpha \rrbracket$;
4. They set $\llbracket v_1 \rrbracket = \alpha \llbracket \text{sk}_{\text{ECDSA}} \rrbracket E_{x_Q} - \llbracket c \rrbracket + \epsilon \llbracket k_s^{-1} \rrbracket E_m - \epsilon \tilde{s} \llbracket 1 \rrbracket$ and broadcast it;
5. They compute $\llbracket v_2 \rrbracket = \llbracket \text{sk}_{\text{ECDSA}} \rrbracket G$ and broadcast it;
6. They compute $\llbracket v_3 \rrbracket = \llbracket k_s^{-1} \rrbracket Q_s$ and broadcast it;
7. They accept iff $v_1 = 0, v_2 = \text{pk}_{\text{ECDSA}}, v_3 = G$.

The first four steps aim to prove the computation of $\tilde{s} = k_s^{-1} (E_m + \text{sk}_{\text{ECDSA}} E_{x_Q})$, the fifth step to prove that $\llbracket \text{sk}_{\text{ECDSA}} \rrbracket$ is a sharing of the ECDSA secret key, and the sixth step to prove the encoded value in $\llbracket k_s^{-1} \rrbracket$.

Apart from the prover-to-verifier communication cost incurred by input sharings (i.e., one auxiliary value for each sharing $\llbracket k_s^{-1} \rrbracket, \llbracket \text{sk}_{\text{ECDSA}} \rrbracket$, and $\llbracket c \rrbracket$), the remaining cost stems solely from the commitments and from broadcasting $\llbracket \alpha \rrbracket$ (i.e. the size of $\llbracket \alpha \rrbracket_{i^*}$, namely the share held by the unopened party i^*).

Note that since V knows $\tilde{s}, G$, and pk_{ECDSA} , they know v_1, v_2 , and v_3 . Hence, the other broadcasts do not add to the communication cost.

The resulting proof size is:

$$2\lambda + \tau (4 \log q + \lambda \log N + 2\lambda) \text{ bits.}$$

Applying the Fiat-Shamir transform yields to a straight-line non-interactive protocol.

7 Instantiation and Performances

Instantiation. We instantiate our generic blind ECDSA (Section 4) by building upon the work developed throughout this paper. Specifically, we use the tailored $\text{NIZK}_{\mathcal{L}_1}$, $\text{NIZK}_{\mathcal{L}_2}$, $\text{NIZK}_{\mathcal{L}_3}$, $\text{NIZK}_{\mathcal{L}_4}$ (Section 5, 6) to instantiate proofs for $\mathcal{L}_1, \mathcal{L}_2, \mathcal{L}_3, \mathcal{L}_4$. We select the Castagnos–Laguillaumie scheme (CL) for the encryption scheme LHE_q and ElGamal scheme (EG) for LHE_p . Concretely, CL is used to encrypt scalars, while EG encrypts elliptic curve points. The commitment schemes Com_p and Com_q are instantiated using the Pedersen commitment (PC) scheme in the corresponding groups. Commitments are vectors of group elements of dimension 1 for PC and ciphertexts are vectors of group elements of dimension 2 for EG and CL, and linear combinations are defined coordinate-wise. We provide definitions of these primitives in A, as well as a full protocol description in E.

The security of our instantiation follows from the security of our generic blind ECDSA, of the tailored NIZKs, of Com_p and Com_q , and the IND-CPA of CL and EG.

Performances. The performance of our blind ECDSA scheme primarily depends on the efficiency of the underlying NIZKs. One chooses the elliptic curve $E(\mathbb{F}_p)$ domain parameters associated with a Koblitz curve `secp256k1`. One fixes the security parameter λ to 2^7 and the number of parties in the MPCitH proofs to 2^8 . The next table summarizes the sizes of individual proofs for each component.

	$\text{NIZK}_{\mathcal{L}_1}$	$\text{NIZK}_{\mathcal{L}_2}$	$\text{NIZK}_{\mathcal{L}_3}$	$\text{NIZK}_{\mathcal{L}_4}$
proof sizes (in kB)	35.29	98.61	35.33	6.10

Hence, the total bandwidth is of 175.33kB. Moreover, since our blind ECDSA scheme produces standard ECDSA signatures, the size of our blind signature is the same as that of an ECDSA signature.

Comparisons. As discussed in the introduction, two previous works proposed blind ECDSA [YL19, QCY21]. As detailed in Appendix B, [YL19] which obtains a bandwidth of 107kB, is not blind since their modified Paillier encryption is insecure. Furthermore, the authors only conjecture the security of their scheme in the one-more unforgeability game, which leaves the security of their construction uncertain. The second proposal [QCY21], on the other hand, is secure when instantiated with the CL encryption scheme, and yields to a bandwidth of 3kB. However, their security assumptions are stronger than ours since they rely on the ABRO model.

Acknowledgments

The authors were supported in part by the French ANR SANGRIA project (ANR-21-CE39-0006), and Project LIONS within dtec.bw, funded by the European Commission under the NextGenerationEU program. We are also grateful to Damien Vergnaud for his valuable insights and advice throughout the development of this work.

References

- [AGH⁺23] Carlos Aguilar-Melchor, Nicolas Gama, James Howe, Andreas Hülsing, David Joseph, and Dongze Yue. The return of the SDitH. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part V*, volume 14008 of *Lecture Notes in Computer Science*, pages 564–596, Lyon,

- France, April 23–27, 2023. Springer, Cham, Switzerland. doi:[10.1007/978-3-031-30589-4_20](https://doi.org/10.1007/978-3-031-30589-4_20).
- [AHIV23] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkatasubramanian. Liger: lightweight sublinear arguments without a trusted setup. *Designs, Codes and Cryptography*, 91(11):3379–3424, 2023. doi:[10.1007/s10623-023-01222-8](https://doi.org/10.1007/s10623-023-01222-8).
- [BBB⁺18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334, San Francisco, CA, USA, May 21–23, 2018. IEEE Computer Society Press. doi:[10.1109/SP.2018.00020](https://doi.org/10.1109/SP.2018.00020).
- [BHLS25] Dan Boneh, Iftach Haitner, Yehuda Lindell, and Gil Segev. Exponent-vrfs and their applications. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 195–224, Cham, 2025. Springer Nature Switzerland. doi:[10.1007/978-3-031-91098-2_8](https://doi.org/10.1007/978-3-031-91098-2_8).
- [Bro05] Daniel R. L. Brown. Generic groups, collision resistance, and ECDSA. *Des. Codes Cryptogr.*, 35(1):119–152, 2005. URL: <https://doi.org/10.1007/s10623-003-6154-z>, doi:[10.1007/S10623-003-6154-Z](https://doi.org/10.1007/S10623-003-6154-Z).
- [CGV14] Jean-Sébastien Coron, Johann Großschädl, and Praveen Kumar Vadnala. Secure conversion between Boolean and arithmetic masking of any order. In Lejla Batina and Matthew Robshaw, editors, *Cryptographic Hardware and Embedded Systems – CHES 2014*, volume 8731 of *Lecture Notes in Computer Science*, pages 188–205, Busan, South Korea, September 23–26, 2014. Springer Berlin Heidelberg, Germany. doi:[10.1007/978-3-662-44709-3_11](https://doi.org/10.1007/978-3-662-44709-3_11).
- [Cha82] David Chaum. Blind signatures for untraceable payments. In David Chaum, Ronald L. Rivest, and Alan T. Sherman, editors, *Advances in Cryptology – CRYPTO’82*, pages 199–203, Santa Barbara, CA, USA, 1982. Plenum Press, New York, USA. doi:[10.1007/978-1-4757-0602-4_18](https://doi.org/10.1007/978-1-4757-0602-4_18).
- [CL15] Guilhem Castagnos and Fabien Laguillaumie. Linearly homomorphic encryption from DDH. In Kaisa Nyberg, editor, *Topics in Cryptology – CT-RSA 2015*, volume 9048 of *Lecture Notes in Computer Science*, pages 487–505, San Francisco, CA, USA, April 20–24, 2015. Springer, Cham, Switzerland. doi:[10.1007/978-3-319-16715-2_26](https://doi.org/10.1007/978-3-319-16715-2_26).
- [CLR24] Sofía Celi, Shai Levin, and Joe Rowell. CDLS: Proving knowledge of committed discrete logarithms with soundness. In Serge Vaudenay and Christophe Petit, editors, *AFRICACRYPT 24: 15th International Conference on Cryptology in Africa*, volume 14861 of *Lecture Notes in Computer Science*, pages 69–93, Douala, Cameroon, July 10–12, 2024. Springer, Cham, Switzerland. doi:[10.1007/978-3-031-64381-1_4](https://doi.org/10.1007/978-3-031-64381-1_4).
- [DOT21] Cyprien Delpech de Saint Guilhem, Emmanuela Orsini, and Titouan Tanguy. Limbo: Efficient zero-knowledge MPCitH-based arguments. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 3022–3036, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press. doi:[10.1145/3460120.3484595](https://doi.org/10.1145/3460120.3484595).
- [ElG85] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985. doi:[10.1109/TIT.1985.1057074](https://doi.org/10.1109/TIT.1985.1057074).

- [Fis05] Marc Fischlin. Communication-efficient non-interactive proofs of knowledge with online extractors. In Victor Shoup, editor, *Advances in Cryptology – CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer Science*, pages 152–168, Santa Barbara, CA, USA, August 14–18, 2005. Springer Berlin Heidelberg, Germany. doi:10.1007/11535218_10.
- [FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part II*, volume 10992 of *Lecture Notes in Computer Science*, pages 33–62, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland. doi:10.1007/978-3-319-96881-0_2.
- [FKP16] Manuel Fersch, Eike Kiltz, and Bertram Poettering. On the provable security of (EC)DSA signatures. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1651–1662, Vienna, Austria, October 24–28, 2016. ACM Press. doi:10.1145/2976749.2978413.
- [FLM22] Armando Faz-Hernández, Watson Ladd, and Deepak Maram. ZKAttest: Ring and group signatures for existing ECDSA keys. In Riham AlTawy and Andreas Hülsing, editors, *SAC 2021: 28th Annual International Workshop on Selected Areas in Cryptography*, volume 13203 of *Lecture Notes in Computer Science*, pages 68–83, Virtual Event, September 29 – October 1, 2022. Springer, Cham, Switzerland. doi:10.1007/978-3-030-99277-4_4.
- [FMRV22] Thibault Feneuil, Jules Maire, Matthieu Rivain, and Damien Vergnaud. Zero-knowledge protocols for the subset sum problem from MPC-in-the-head with rejection. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology – ASIACRYPT 2022, Part II*, volume 13792 of *Lecture Notes in Computer Science*, pages 371–402, Taipei, Taiwan, December 5–9, 2022. Springer, Cham, Switzerland. doi:10.1007/978-3-031-22966-4_13.
- [FR23] Thibault Feneuil and Matthieu Rivain. Threshold linear secret sharing to the rescue of MPC-in-the-head. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part I*, volume 14438 of *Lecture Notes in Computer Science*, pages 441–473, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore. doi:10.1007/978-981-99-8721-4_14.
- [FR25] Thibault Feneuil and Matthieu Rivain. Threshold computation in the head: Improved framework for post-quantum signatures and zero-knowledge arguments. *Journal of Cryptology*, 38, 06 2025. doi:10.1007/s00145-025-09543-8.
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology – CRYPTO’86*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194, Santa Barbara, CA, USA, August 1987. Springer Berlin Heidelberg, Germany. doi:10.1007/3-540-47721-7_12.
- [Fs24] Matteo Frigo and abhi shelat. Anonymous credentials from ECDSA. Cryptology ePrint Archive, Report 2024/2010, 2024. URL: <https://eprint.iacr.org/2024/2010>.
- [FW24] Georg Fuchsbauer and Mathias Wolf. Concurrently secure blind schnorr signatures. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part II*, volume 14652 of *Lecture Notes in Computer*

- Science*, pages 124–160, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland. doi:[10.1007/978-3-031-58723-8_5](https://doi.org/10.1007/978-3-031-58723-8_5).
- [GMR88] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM J. Comput.*, 17(2):281–308, 1988. doi:[10.1137/0217017](https://doi.org/10.1137/0217017).
- [GMR89] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.
- [GS22] Jens Groth and Victor Shoup. On the security of ECDSA with additive key derivation and presignatures. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part I*, volume 13275 of *Lecture Notes in Computer Science*, pages 365–396, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland. doi:[10.1007/978-3-031-06944-4_13](https://doi.org/10.1007/978-3-031-06944-4_13).
- [HK23] Dominik Hartmann and Eike Kiltz. Limits in the provable security of ECDSA signatures. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023: 21st Theory of Cryptography Conference, Part IV*, volume 14372 of *Lecture Notes in Computer Science*, pages 279–309, Taipei, Taiwan, November 29 – December 2, 2023. Springer, Cham, Switzerland. doi:[10.1007/978-3-031-48624-1_11](https://doi.org/10.1007/978-3-031-48624-1_11).
- [HKL21] David Heath, Vladimir Kolesnikov, and Jiahui Lu. Efficient generic arithmetic for kkw. In Shlomi Dolev, Oded Margalit, Benny Pinkas, and Alexander Schwarzmann, editors, *Cyber Security Cryptography and Machine Learning*, pages 414–431, Cham, 2021. Springer International Publishing. doi:[10.1007/978-3-030-78086-9_31](https://doi.org/10.1007/978-3-030-78086-9_31).
- [IKOS07] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge from secure multiparty computation. In David S. Johnson and Uriel Feige, editors, *39th Annual ACM Symposium on Theory of Computing*, pages 21–30, San Diego, CA, USA, June 11–13, 2007. ACM Press. doi:[10.1145/1250790.1250794](https://doi.org/10.1145/1250790.1250794).
- [IKOS09] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge proofs from secure multiparty computation. *SIAM J. Comput.*, 39(3):1121–1152, 2009. doi:[10.1007/978-3-030-78086-9_31](https://doi.org/10.1007/978-3-030-78086-9_31).
- [Jou23] Antoine Joux. MPC in the head for isomorphisms and group actions. Cryptology ePrint Archive, Report 2023/664, 2023. URL: <https://eprint.iacr.org/2023/664>.
- [KKW18] Jonathan Katz, Vladimir Kolesnikov, and Xiao Wang. Improved non-interactive zero knowledge with applications to post-quantum signatures. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 525–537, Toronto, ON, Canada, October 15–19, 2018. ACM Press. doi:[10.1145/3243734.3243805](https://doi.org/10.1145/3243734.3243805).
- [MV23] Jules Maire and Damien Vergnaud. Commitments with efficient zero-knowledge arguments from subset sum problems. In Gene Tsudik, Mauro Conti, Kaitai Liang, and Georgios Smaragdakis, editors, *ESORICS 2023: 28th European Symposium on Research in Computer Security, Part I*, volume 14344 of *Lecture Notes in Computer Science*, pages 189–208, The Hague, The Netherlands,

- September 25–29, 2023. Springer, Cham, Switzerland. doi:10.1007/978-3-031-50594-2_10.
- [MV25] Jules Maire and Damien Vergnaud. Compact zero-knowledge arguments for blum integers. *Theor. Comput. Sci.*, 1038:115155, 2025. URL: <https://doi.org/10.1016/j.tcs.2025.115155>, doi:10.1016/J.TCS.2025.115155.
- [Ped92] Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In Joan Feigenbaum, editor, *Advances in Cryptology – CRYPTO’91*, volume 576 of *Lecture Notes in Computer Science*, pages 129–140, Santa Barbara, CA, USA, August 11–15, 1992. Springer Berlin Heidelberg, Germany. doi:10.1007/3-540-46766-1_9.
- [PS00] David Pointcheval and Jacques Stern. Security arguments for digital signatures and blind signatures. *Journal of Cryptology*, 13(3):361–396, June 2000. doi:10.1007/s001450010003.
- [PV05] Pascal Paillier and Damien Vergnaud. Discrete-log-based signatures may not be equivalent to discrete log. In Bimal K. Roy, editor, *Advances in Cryptology – ASIACRYPT 2005*, volume 3788 of *Lecture Notes in Computer Science*, pages 1–20, Chennai, India, December 4–8, 2005. Springer Berlin Heidelberg, Germany. doi:10.1007/11593447_1.
- [QCY21] Xianrui Qin, Cailing Cai, and Tsz Hon Yuen. One-more unforgeability of blind ECDSA. In Elisa Bertino, Haya Shulman, and Michael Waidner, editors, *ESORICS 2021: 26th European Symposium on Research in Computer Security, Part II*, volume 12973 of *Lecture Notes in Computer Science*, pages 313–331, Darmstadt, Germany, October 4–8, 2021. Springer, Cham, Switzerland. doi:10.1007/978-3-030-88428-4_16.
- [Sch91] Claus-Peter Schnorr. Efficient signature generation by smart cards. *Journal of Cryptology*, 4(3):161–174, January 1991. doi:10.1007/BF00196725.
- [WOS⁺25] Anna P. Y. Woo, Alex Ozdemir, Chad Sharp, Thomas Pornin, and Paul Grubbs. Efficient proofs of possession for legacy signatures. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 3291–3308, 2025. doi:10.1109/SP61157.2025.00080.
- [YL19] Xun Yi and Kwok-Yan Lam. A new blind ECDSA scheme for bitcoin transaction anonymity. In Steven D. Galbraith, Giovanni Russello, Willy Susilo, Dieter Gollmann, Engin Kirda, and Zhenkai Liang, editors, *ASIACCS 19: 14th ACM Symposium on Information, Computer and Communications Security*, pages 613–620, Auckland, New Zealand, July 9–12, 2019. ACM Press. doi:10.1145/3321705.3329816.
- [Zip79] Richard Zippel. Probabilistic algorithms for sparse polynomials. In Edward W. Ng, editor, *Symbolic and Algebraic Computation*, pages 216–226, Berlin, Heidelberg, 1979. Springer Berlin Heidelberg. doi:10.1007/3-540-09519-5_73.

A Definitions

Elliptic curves. Let p be a prime number. An elliptic curve E over $\mathbb{F}_p$ is defined by an equation of the form

$$y^2 = x^3 + ax + b,$$

where $a, b \in \mathbb{F}_p$, and the curve is required to have no singular points, i.e., the discriminant $\Delta = 4a^3 + 27b^2 \neq 0$. The set of points, including the point at infinity $\mathcal{O}$, forms an Abelian group under the operation of point addition. For two points $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ different from $\mathcal{O}$ with $P \neq \pm Q$, the coordinates of the sum $R = P + Q = (x_3, y_3)$ are given by

$$x_3 = \lambda^2 - x_1 - x_2 \bmod p, \quad y_3 = \lambda(x_1 - x_3) - y_1 \bmod p, \quad \text{where } \lambda = \frac{y_2 - y_1}{x_2 - x_1} \bmod p.$$

A.1 Digital Signatures

A *digital signature scheme* is a tuple of PPT algorithms (**Setup**, **KeyGen**, **Sign**, **Verify**) such that:

- **Setup**(1^λ) $\rightarrow$ **pp**. On input λ , the *setup* algorithm outputs the public parameters **pp** containing a description of the message space $\mathcal{M}$.
- **KeyGen**(**pp**) $\rightarrow$ (**sk**, **pk**). On input **pp**, the *key generation* algorithm outputs a key pair (**sk**, **pk**).
- **Sign**(m , **sk**) $\rightarrow$ σ . On input a message $m \in \mathcal{M}$ and a signing key **sk**, the *signing* algorithm outputs a bit string σ .
- **Verify**(m , σ , **pk**) $\rightarrow b \in \{0, 1\}$. On input a message, a bit string σ and a public key **pk**, the *verification* algorithm outputs a bit b . If $b = 1$, then σ is called a *signature* of m for **pk**.

Moreover, it satisfies the following correctness property: we have for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Verify}(m, \sigma, \text{pk}) = 1 \mid \begin{array}{l} \text{pp} \xleftarrow{\$} \text{Setup}(1^\lambda), m \xleftarrow{\$} \mathcal{M}, \\ (\text{sk}, \text{pk}) \xleftarrow{\$} \text{KeyGen}(\text{pp}), \sigma \xleftarrow{\$} \text{Sign}(m, \text{sk}) \end{array} \right] = 1.$$

The security of a digital signature scheme is defined with the following property.

Definition 3. Let $t, q_S : \mathbb{N} \rightarrow \mathbb{N}$ and $\epsilon : \mathbb{N} \rightarrow [0, 1]$. A digital signature scheme (**Setup**, **KeyGen**, **Sign**, **Verify**) is said (t, q_S, ϵ) -*unforgeable under chosen message attacks* (EUF-CMA) if for all algorithms **A**, we have for all $\lambda \in \mathbb{N}$:

$$\Pr \left[\begin{array}{l} m^* \text{ not queried to } \text{Sign}(\cdot, \text{sk}) \text{ and} \\ \text{Verify}(m^*, \sigma^*, \text{pk}) = 1 \end{array} \mid \begin{array}{l} \text{pp} \xleftarrow{\$} \text{Setup}(1^\lambda), \\ (\text{sk}, \text{pk}) \xleftarrow{\$} \text{KeyGen}(\text{pp}), \\ (m^*, \sigma^*) \xleftarrow{\$} \text{A}^{\text{Sign}(\cdot, \text{sk})}(\text{pk}) \end{array} \right] \leq \epsilon(\lambda)$$

when **A** runs in time at most $t(\lambda)$ and makes at most $q_S(\lambda)$ queries to its signing oracle.

A.2 Commitment Schemes

A *commitment scheme* is a triple of PPT algorithms (**Setup**, **Com**, **Ver**) such that:

- **Setup**(1^λ) $\rightarrow$ **pp**. On input λ , the *setup* algorithm outputs the public parameters **pp** containing a description of the message space $\mathcal{M}$.
- **Com**(**pp**, m) $\rightarrow$ (c , **aux**). On input **pp** and $m \in \mathcal{M}$, the *commit* algorithm outputs a commitment-opening pair (c , **aux**).
- **Ver**(**pp**, m , c , **aux**) $\rightarrow b \in \{0, 1\}$. On input **pp**, $m \in \mathcal{M}$ and (c , **aux**), the *verifying* (or *opening*) algorithm outputs a bit $b \in \{0, 1\}$.

Moreover, it satisfies the following correctness property: we have for all $\lambda \in \mathbb{N}$,

$$\Pr[\text{Ver}(\text{pp}, m, c, \text{aux}) = 1 \mid \text{pp} \xleftarrow{\$} \text{Setup}(1^\lambda), m \xleftarrow{\$} \mathcal{M}, (c, \text{aux}) \xleftarrow{\$} \text{Com}(\text{pp}, m)] = 1.$$

There are two security notions underlying a commitment scheme.

Definition 4. Let $t : \mathbb{N} \rightarrow \mathbb{N}$ and $\epsilon : \mathbb{N} \rightarrow [0, 1]$. A commitment scheme $(\text{Setup}, \text{Com}, \text{Ver})$ is said:

- (t, ϵ) -*computationally hiding* if for all two-phase algorithms $A = (A_1, A_2)$, we have for all $\lambda \in \mathbb{N}$:

$$\Pr \left[b = b' \mid \begin{array}{l} \text{pp} \xleftarrow{\$} \text{Setup}(1^\lambda), (m_0, m_1, s) \xleftarrow{\$} A_1(\text{pp}), b \xleftarrow{\$} \{0, 1\} \\ (c, \text{aux}) \xleftarrow{\$} \text{Com}(\text{pp}, m_b), b' \xleftarrow{\$} A_2(c, s) \end{array} \right] \leq \frac{1}{2} + \epsilon(\lambda)$$

when A runs in time at most $t(\lambda)$.

- (t, ϵ) -*computationally binding* if for all algorithms A , we have for all $\lambda \in \mathbb{N}$:

$$\Pr \left[\begin{array}{l} m_1 \neq m_2 \\ \text{Ver}(\text{pp}, m_1, c, \text{aux}_1) = 1 \\ \text{Ver}(\text{pp}, m_2, c, \text{aux}_2) = 1 \end{array} \mid \begin{array}{l} \text{pp} \xleftarrow{\$} \text{Setup}(1^\lambda), \\ (m_1, m_2, \text{aux}_1, \text{aux}_2, c) \xleftarrow{\$} A(\text{pp}) \end{array} \right] \leq \epsilon(\lambda)$$

when A runs in time at most $t(\lambda)$.

Pedersen commitment. The Pedersen commitment scheme (PC) [Ped92] is a perfectly hiding and computationally binding commitment scheme for the message space of scalars $\mathbb{Z}_q$. It is defined as follows:

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$: Given G and H two generators of a cyclic group $\mathbb{G}$ of prime order q , the *setup* algorithm outputs the public parameters $\text{pp} = (\mathbb{G}, q, G, H)$ and $\mathcal{M} = \mathbb{Z}_q$.
- $\text{Com}(\text{pp}, m) \rightarrow (C, r)$: On input pp and $m \in \mathbb{Z}_q$, the *commit* algorithm outputs a random scalar $r \in \mathbb{Z}_q$ and the commitment $C = mG + rH$.
- $\text{Ver}(\text{pp}, m, C, r) \rightarrow b$: On input pp , $m \in \mathbb{Z}_q$ and (C, r) , the *verifying* algorithm outputs $b = 1$ if $C = mG + rH$, and $b = 0$ otherwise.

A.3 Encryption Schemes

A public-key encryption scheme is a quadruple of PPT algorithms $(\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ such that:

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$: On input the security parameter $\lambda \in \mathbb{N}$, the *setup* algorithm outputs the public parameters pp , which include a description of the message space $\mathcal{M}$.
- $\text{KeyGen}(\text{pp}) \rightarrow (\text{pk}, \text{sk})$: On input pp , the *key generation* algorithm outputs a public/secret key pair (pk, sk) .
- $\text{Enc}(\text{pp}, \text{pk}, m) \rightarrow c$: On input pp , a public key pk , and a message $m \in \mathcal{M}$, the *encryption* algorithm outputs a ciphertext c .
- $\text{Dec}(\text{pp}, \text{sk}, c) \rightarrow m$ or $\perp$: On input pp , a secret key sk , and a ciphertext c , the *decryption* algorithm outputs a message $m \in \mathcal{M}$ or $\perp$ in case of decryption failure.

Moreover, it satisfies the following correctness property: we have for all $\lambda \in \mathbb{N}$:

$$\Pr \left[\text{Dec}(\text{pp}, \text{sk}, \text{Enc}(\text{pp}, \text{pk}, m)) = m \mid \begin{array}{l} \text{pp} \xleftarrow{\$} \text{Setup}(1^\lambda), \\ (\text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}(\text{pp}), \\ m \xleftarrow{\$} \mathcal{M} \end{array} \right] = 1.$$

The security of an encryption scheme is defined with the following property.

Definition 5. Let $t : \mathbb{N} \rightarrow \mathbb{N}$ and $\epsilon : \mathbb{N} \rightarrow [0, 1]$. The encryption scheme is said to be (t, ϵ) -IND-CPA (*indistinguishability under chosen-plaintext attack*) if for all two-phase adversaries $A = (A_1, A_2)$ running in time at most $t(\lambda)$, we have:

$$\Pr \left[b = b' \mid \begin{array}{l} \text{pp} \xleftarrow{\$} \text{Setup}(1^\lambda), (\text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}(\text{pp}), \\ (m_0, m_1, s) \xleftarrow{\$} A_1(\text{pp}, \text{pk}), b \xleftarrow{\$} \{0, 1\}, \\ c \xleftarrow{\$} \text{Enc}(\text{pp}, \text{pk}, m_b), b' \xleftarrow{\$} A_2(c, s) \end{array} \right] \leq \frac{1}{2} + \epsilon(\lambda).$$

ElGamal encryption. The ElGamal encryption scheme (EG) [ElG85] is a public-key encryption scheme over a group of prime order that is additively homomorphic and IND-CPA secure under the decisional Diffie-Hellman (DDH) assumption. It is defined as follows:

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$: Given a generator G of a cyclic group $\mathbb{G}$ of prime order q , the *setup* algorithm outputs the public parameters $\text{pp} = (\mathbb{G}, q, G)$ and $\mathcal{M} = \mathbb{G}$.
- $\text{KeyGen}(\text{pp}) \rightarrow (\text{pk}, \text{sk})$: On input pp , the *key generation* algorithm samples a secret key $\text{sk} \in \mathbb{Z}_q$ uniformly at random and computes the public key $\text{pk} = \text{sk} \cdot G$.
- $\text{Enc}(\text{pp}, \text{pk}, M) \rightarrow C$: On input pp , a public key pk , and a message $M \in \mathbb{G}$, the *encryption* algorithm samples $r \in \mathbb{Z}_q$ uniformly at random and computes the ciphertext $C = (C_1, C_2) = (r \cdot G, M + r \cdot \text{pk})$.
- $\text{Dec}(\text{pp}, \text{sk}, C) \rightarrow M$: On input pp , a secret key sk , and a ciphertext C , the *decryption* algorithm parses $C = (C_1, C_2)$ and computes $M = C_2 - \text{sk} \cdot C_1$.

Castagnos-Laguillaumie encryption. The Castagnos-Laguillaumie encryption scheme (CL) [CL15] specifies two algorithms, CL.Gen and CL.Solve. CL.Gen takes a computational security parameter 1^λ and a prime $q > 2^\lambda$, and outputs a tuple

$$\text{pp} := (q, \bar{s}, f, g_q, \hat{G}, F; \rho) \leftarrow \text{CL.Gen}(1^\lambda, q),$$

where ρ denotes the randomness used by CL.Gen. Here, $(\hat{G}, \cdot)$ is a finite abelian group of order $\hat{n} := q \cdot \hat{s}$. The factor $\hat{s}$ remains unknown, but we are given an upper bound $\bar{s} \geq \hat{s}$. Moreover, $\gcd(q, \hat{s}) = 1$, and the size of $\hat{s}$ depends on λ . Let $F = \langle f \rangle \subset \hat{G}$ be the unique subgroup of order q , and let $G \subset \hat{G}$ be the cyclic subgroup of order $n := q \cdot s$ (elements of G are less efficiently recognizable than those of $\hat{G}$). Denoting $G_q = \langle g_q \rangle$ the subgroup of q -th powers, then G factors as $G \cong G_q \times F$ with $g := g_q \cdot f$ being a generator of G . Additionally, the given upper bound $\bar{s}$ is chosen such that the distribution of $\{xG \mid x \in \{0, \dots, \bar{s}q/(4\delta) - 1\}\}$ has statistical distance at most δ to the uniform distribution over G . For simplicity, let us denote $\{0, \dots, Bq - 1\}$ such interval.

Given the output of CL.Gen, the second algorithm CL.Solve deterministically and efficiently solves the discrete logarithm problem in the subgroup F .

As for the EG encryption scheme, the security of the CL encryption scheme relies on the DDH assumption.

Given CL.Gen and CL.Solve, CL can be defined as follows.

- $\text{CL.KeyGen}(1^\lambda, q) \rightarrow (\text{pk}_{\text{CL}}, \text{sk}_{\text{CL}})$: On input $(1^\lambda, q)$, the *key generation* algorithm calls to $\text{pp} \leftarrow \text{CL.Gen}(1^\lambda, q)$, and outputs $\text{sk}_{\text{CL}} \xleftarrow{\$} \{0, \dots, Bq - 1\}$ and $\text{pk}_{\text{CL}} = (q, \bar{s}, f, g, h)$, where $h = \text{sk}_{\text{CL}} \cdot g$.
- $\text{CL.Enc}(\text{pk}_{\text{CL}}, m) \rightarrow (c_1, c_2)$: On input the public key pk_{CL} and a message $m \in \mathbb{Z}_q$, it outputs the ciphertext $(c_1, c_2) = (r \cdot g, m \cdot f + r \cdot h)$, where $r \xleftarrow{\$} \{0, \dots, Bq - 1\}$.
- $\text{CL.Dec}(\text{pk}_{\text{CL}}, \text{sk}_{\text{CL}}, (c_1, c_2)) \rightarrow m$: On input the keys and a ciphertext (c_1, c_2) , it computes $c_2 - \text{sk}_{\text{CL}} \cdot c_1 = m \cdot f$ and calls to CL.Solve on $m \cdot f$ to get m .

We may also use some homomorphic properties:

- $\text{CL.Sum}(\text{pk}_{\text{CL}}, (c_1, c_2), (c'_1, c'_2)) \rightarrow (c''_1, c''_2)$: On input the public key and two ciphertexts $(c_1, c_2), (c'_1, c'_2)$ for messages $m_1, m_2 \in \mathbb{Z}_q$, it outputs a ciphertext $(c''_1, c''_2) = (c_1 + c'_1, c_2 + c'_2)$ for the message $m_1 + m_2$.
- $\text{CL.Scal}(\text{pk}_{\text{CL}}, (c_1, c_2), \alpha) \rightarrow (c'_1, c'_2)$: On input the public key, a ciphertext (c_1, c_2) for a message $m \in \mathbb{Z}_q$, and a scalar $\alpha \in \mathbb{Z}_q$, it outputs a ciphertext $(c'_1, c'_2) = (\alpha c_1, \alpha c_2)$ for the message αm .

B Blind ECDSA from [YL19] is insecure

The blind ECDSA signature scheme proposed by Yi and Lam [YL19] is built on the following variant of the Paillier cryptosystem. Given a large prime q (a public parameter of ECDSA), the modified Paillier cryptosystem is described as follows:

KeyGen: A user randomly chooses two large distinct primes p_1, p_2 , such that $\gcd(p_1 - 1, q) = 1$ and $\gcd(p_2 - 1, q) = 1$, and computes

$$N = qp_1p_2 \tag{2}$$

$$g = (1 + N)^{p_1p_2} \bmod N^2 \tag{3}$$

and publishes the public key (N, g) , and keeps the private key (p_1, p_2) secret.

Enc: Given the public key (N, g) of the user, one can encrypt a message m , where m is a positive integer less than q , by randomly choosing r from $\mathbb{Z}_{N^2}$ and computing

$$C = g^m r^N \bmod N^2 \tag{4}$$

where C is the ciphertext of m .

Dec: The user can decrypt the ciphertext C with the private key (p_1, p_2) by computing

$$D = C^{(p_1-1)(p_2-1)(q-1)} \bmod N^2 \tag{5}$$

$$m' = [(D - 1)/(Np_1p_2)][(p_1 - 1)(p_2 - 1)(q - 1)]^{-1} \bmod q \tag{6}$$

In [YL19, Theorem 4], Yi and Lam claimed that this encryption scheme is semantically secure assuming the computational hardness of distinguishing N -th residues (i.e. elements $z \in \mathbb{Z}_{N^2}$ for which there exist $y \in \mathbb{Z}_{N^2}$ such that $z = y^N \bmod N^2$) from random elements modulo N^2 . Their claim is flawed.

Indeed, from the previous description, we know that q (the public value related to ECDSA) divides the integer N and thus q^2 divides N^2 . We can compute

$$C^{q-1} \bmod q^2 = g^{m(q-1)} r^{(q-1)N} \bmod q^2 \tag{7}$$

from (4). Since $N = qp_1p_2$, we have

$$r^{(q-1)N} = r^{(q-1)qp_1p_2} = \left(r^{q(q-1)}\right)^{p_1p_2} = 1 \bmod q^2 \quad (8)$$

since $|\mathbb{Z}_{q^2}^*| = q(q-1)$. We thus have

$$C^{q-1} \bmod q^2 = g^{m(q-1)} = h^m \bmod q^2 \quad (9)$$

where $h = g^{q-1} \bmod q^2$. The base h is of order ρ , a proper divisor $(q-1) \bmod q^2$ since $h^{(q-1)} = 1 \bmod q^2$. From (2), we know that

$$g = (1 + N)^{p_1p_2} \bmod N^2$$

and

$$\begin{aligned} h = g^{q-1} &= (1 + qp_1p_2)^{(q-1)p_1p_2} \bmod q^2 = 1 + (p_1^2p_2^2)q(q-1) \bmod q^2 \\ &= 1 - qp_1^2p_2^2 \bmod q^2 \end{aligned} \quad (10)$$

Since p_1 and p_2 are primes different from q , we get that $h \neq 1 \bmod q^2$ and thus $\rho > 1$. To break the semantic security of the variant of Paillier's encryption scheme, an adversary can simply pick two messages m_0 and m_1 such that $m_0 \neq m_1 \bmod r$ (for instance $m_0 = 0$ and $m_1 = 1$). From a ciphertext C corresponding to one of these two plaintexts m_b for $b \in \{0, 1\}$, it computes $C^{q-1} \bmod q^2$. From (9), it obtains $h^{m_b} \bmod q^2$ and it can simply compute the two (different) values $h^{m_0} \bmod q^2$ and $h^{m_1} \bmod q^2$ to obtain, with probability 1, which plaintext has been encrypted.

C Theorems Proof

C.1 Lemmas

Lemma 1 (Splitting lemma in multi-dimension [PS00]). *Let $X_1, \dots, X_d$ be finite sets, and let $A \subset X_1 \times \dots \times X_d$ such that for any uniformly and independently chosen elements $x_1, \dots, x_d$,*

$$\Pr[(x_1, \dots, x_d) \in A] = \epsilon .$$

For any $\alpha < \epsilon$, define

$$B_\alpha = \{(x_1, \dots, x_d) \in X_1 \times \dots \times X_d \mid \Pr_{(x'_2, \dots, x'_d) \in X_2 \times \dots \times X_d}[(x_1, x'_2, \dots, x'_d) \in A] \geq \epsilon - \alpha\},$$

then

1. $\Pr[B_\alpha] \geq \alpha$
2. $\Pr[B_\alpha | A] \geq \alpha / \epsilon$.

Lemma 2 (Schwartz-Zippel lemma [Zip79]). *Let $\mathbb{F}$ be a field, $p \in \mathbb{F}[x_1, \dots, x_n]$ be a (non-zero) multivariate polynomial of degree $d \geq 0$, and $\mathbb{S} \subseteq \mathbb{F}$ be a (non-empty) finite subset. Then, for any subset of elements $s_1, \dots, s_n$ uniformly and independently chosen from $\mathbb{S}$,*

$$\Pr[p(s_1, \dots, s_n) = 0] \leq \frac{d}{|\mathbb{F}|} .$$

C.2 Proof of Theorem 1

Proof. Completeness. If P knows a valid witness $(\{x_i, r_i, s_i, t_i, u_i, v_i, w_i, \alpha_i, \beta_i\}_{i \in [2]}, \beta_3)$, and if the computation described in **SubAddPoint** is honestly performed, then all the checks of V pass. The completeness is hence perfect.

Honest-verifier zero-knowledge. We build a PPT simulator $\mathcal{S}$ that outputs transcripts that are indistinguishable from real transcripts without knowing a valid witness. $\mathcal{S}$ takes as input $\{X_i, Y_i, Z_i, U_i, V_i, W_i\}_{i \in [2]}$, and works as follows:

1. Sample at random:

$$\begin{aligned} \gamma_1, \gamma_2, \gamma_3, \gamma_4, c &\stackrel{\$}{\leftarrow} \mathbb{S}, \\ a_1, a_2, a_3, a_4, b_1, b_2 &\stackrel{\$}{\leftarrow} \mathbb{F}_p, \\ A_{4,2} &\stackrel{\$}{\leftarrow} E'(\mathbb{F}). \end{aligned}$$

2. Compute $\{B_i, C_i\}_{i \in [2]}$ and W_3 as in **SubAddPoint**.

3. Compute the commitments:

$$\begin{aligned} A_1 &= \text{Com}_p(a_1; b_1) - cX_1, \\ A_2 &= \text{Com}_p(a_2; b_2) - cX_2, \\ A_3 &= \text{Com}_p(0; a_3) - cW_3, \\ A_{4,1} &= a_1B_1 + \gamma_4a_2B_2 - A_{4,2}\gamma_4 - (C_1 + C_2\gamma_4)c - \text{Com}_p(0; a_4). \end{aligned}$$

4. Output the transcript:

$$(A_1, A_2, A_3, A_{4,1}, A_{4,2}, a_1, a_2, a_3, a_4, b_1, b_2) .$$

Hence, $\mathcal{S}$ outputs a transcript that is identically distributed to the genuine transcript without knowing the witness.

Knowledge soundness. Let us first show how to extract a valid witness

$$(\{x_i, r_i, s_i, t_i, u_i, v_i, w_i, \alpha_i, \beta_i\}_{i \in [2]}, \beta_3)$$

from eight transcripts satisfying some specific conditions. We will then show how to get such transcripts from a rewindable black-box access to $\tilde{P}$, which denotes a malicious prover.

Transcripts used for extraction. Let us denote the eight transcripts as

$$T_i := (\text{Com}_{1,T_i}, \text{Chall}_{1,T_i}, \text{Com}_{2,T_i}, \text{Chall}_{2,T_i}, \text{Resp}_{T_i}) \text{ for } i \in [8] ,$$

where

$$\begin{aligned} \text{Com}_{1,T_i} &= (A_{1,T_i}, A_{2,T_i}, A_{3,T_i}), \text{Chall}_{1,T_i} = (\gamma_{1,T_i}, \gamma_{2,T_i}, \gamma_{3,T_i}), \\ \text{Com}_{2,T_i} &= (A_{4,1,T_i}, A_{4,2,T_i}), \text{Chall}_{2,T_i} = (c_{T_i}, \gamma_{4,T_i}), \\ \text{Resp}_{T_i} &= (a_{1,T_i}, a_{2,T_i}, a_{3,T_i}, a_{4,T_i}, b_{1,T_i}, b_{2,T_i}) . \end{aligned}$$

Let us assume that we can extract eight transcripts $\{T_i\}_{i \in [8]}$ from $\tilde{P}$ such that $\text{Com}_{1,T_1} = \text{Com}_{1,T_i}$ for $i \in [8]$. Moreover, we add the following constraints:

- $c_{T_1} \neq c_{T_2}$, then one can extract $x_1, x_2, r_1, r_2, \alpha_1, \alpha_2, \beta_1, \beta_2$.
- $\gamma_{3,T_1} = \gamma_{3,T_2}$, then one can extract $w_1 + \gamma_{3,T_1} w_2$.

- $c_{T_3} \neq c_{T_4}$ and $\gamma_{3,T_1} \neq \gamma_{3,T_3} = \gamma_{3,T_4}$, then one can extract $w_1 + \gamma_{3,T_3}w_2$ and thus w_1, w_2 .
- $\gamma_{1,T_1} = \gamma_{1,T_2}$, $\gamma_{2,T_1} = \gamma_{2,T_2}$, $\gamma_{4,T_1} = \gamma_{4,T_2}$, then one can extract $x_1(s_1 + \gamma_{1,T_1}t_1) + \gamma_{4,T_1}x_2(s_2 + t_2\gamma_{2,T_1})$ and $u_1 + \gamma_{1,T_1}v_1 + \gamma_{4,T_1}(u_2 + \gamma_{2,T_1}v_2)$
- Identically, with the conditions $\gamma_{1,T_3} = \gamma_{1,T_4}$, $\gamma_{2,T_3} = \gamma_{2,T_4}$, $\gamma_{4,T_3} = \gamma_{4,T_4}$, one can extract $x_1(s_1 + \gamma_{1,T_3}t_1) + \gamma_{4,T_3}x_2(s_2 + t_2\gamma_{2,T_3})$ and $u_1 + \gamma_{1,T_3}v_1 + \gamma_{4,T_3}(u_2 + \gamma_{2,T_3}v_2)$.
- If $\gamma_{4,T_1} \neq \gamma_{4,T_3}$, $\gamma_{1,T_1} = \gamma_{1,T_3}$, $\gamma_{2,T_1} = \gamma_{2,T_3}$, then one can extract $s_1 + \gamma_{1,T_1}t_1$, $s_2 + t_2\gamma_{2,T_1}$, $u_1 + \gamma_{1,T_1}v_1$, and $u_2 + \gamma_{2,T_1}v_2$.
- Choose T_5 such that $\gamma_{1,T_1} \neq \gamma_{1,T_5}$ and $\gamma_{2,T_1} \neq \gamma_{2,T_5}$. If one chooses T_6, T_7, T_8 according to T_5 as one has chosen T_2, T_3, T_4 according to T_1 , then one can extract $s_1 + \gamma_{1,T_5}t_1$, $s_2 + t_2\gamma_{2,T_5}$, $u_1 + \gamma_{1,T_5}v_1$, and $u_2 + \gamma_{2,T_5}v_2$. Finally, one can extract $s_1, s_2, t_1, t_2, u_1, u_2, v_1, v_2$.

Extraction from $\tilde{\mathbf{P}}$. Now, let us analyze how we can get such transcripts.

Let K be the set of randomness in **SubAddPoint** and let $\text{Succ}_{\tilde{\mathbf{P}}} \subset K^8$ be the set of randomness from which one can generate eight different successful transcripts of **SubAddPoint**. By hypothesis, we have $\Pr[(k_1, \dots, k_8) \in \text{Succ}_{\tilde{\mathbf{P}}}] = \tilde{\epsilon}$. We shall further denote by r_{Com} the randomness of $\tilde{\mathbf{P}}$ which is used to generate the initial commitment Com_{1,T_1} . Let us fix an arbitrary value $\alpha \in (0, 1)$ such that $(1 - \alpha)\tilde{\epsilon} > 6\epsilon$ (assuming that $\tilde{\epsilon} > 6\epsilon$). Let κ be a possible realization of K . We say that κ is good if it is such that

$$\Pr[(\kappa, k_2, \dots, k_8) \in \text{Succ}_{\tilde{\mathbf{P}}} | \kappa \in K] \geq (1 - \alpha)\tilde{\epsilon}.$$

To obtain all the transcripts, we run the protocol a first time to get T_1 . Using the randomness of T_1 , we can get further successful transcripts with high probability by rewinding the protocol after the initial commitment. One formally describes the extractor routine below, where η is a parameter that will be specified further in the proof. A set of eight transcripts is said to be *extractable* if it satisfies the previously stated conditions required for transcripts to be used in extraction.

Extractor $\mathbf{E}_{\text{AddPoint}}$:

1. Repeat $+\infty$ times
2. Run $\tilde{\mathbf{P}}$ with honest $\mathbf{V}$ to get T_1 , go to step 1 if T_1 is unsuccessful
3. Do η times
4. Run $\tilde{\mathbf{P}}$ with same r_{Com} as T_1 to get seven more transcripts
5. If the seven transcripts are succesfull and extractable
6. Return the eight transcripts

Let us evaluate the probability that $\mathbf{E}_{\text{AddPoint}}$ outputs such transcripts. Consider a loop iteration during which eight successful transcripts have been found. We analyze the probability that these transcripts are also extractable, and let E be the event *the eight transcripts are extractable*. Hence, $\Pr(\bar{E})$ depends on the matrix

$$A = \begin{bmatrix} 1 & \gamma_{1,T_1} & \gamma_{4,T_1} & \gamma_{4,T_1}\gamma_{2,T_1} & c_{T_1} & c_{T_1}\gamma_{1,T_1} & c_{T_1}\gamma_{4,T_1} & c_{T_1}\gamma_{4,T_1}\gamma_{2,T_1} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & \gamma_{1,T_8} & \gamma_{4,T_8} & \gamma_{4,T_8}\gamma_{2,T_8} & c_{T_8} & c_{T_8}\gamma_{1,T_8} & c_{T_8}\gamma_{4,T_8} & c_{T_8}\gamma_{4,T_8}\gamma_{2,T_8} \end{bmatrix},$$

which highlights the equations behind the conditions previously stated for transcripts to be used in extraction. More precisely, the probability $\Pr(\bar{E})$ can be bounded using the

Schwart-Zippel Lemma 2 as

$$\Pr(\bar{E}) \leq \frac{\deg(\det(A))}{|\mathbb{S}|} = \frac{12}{|\mathbb{S}|}, \quad (11)$$

since the challenges are randomly picked in $\mathbb{S}$, and $\det(A)$ is a polynomial of degree 12.

Therefore, the probability that we obtain eight successful and *extractable* transcripts is then

$$\begin{aligned} & \Pr\left(\bigcap_{i \in [8]} \{T_i \text{ is successful}\} \cap E | r_{\text{Com}} \text{ is good}\right) \\ &= \Pr\left(\bigcap_{i \in [8]} \{T_i \text{ is succ.}\} | r_{\text{Com}} \text{ is good}\right) - \Pr\left(\bigcap_{i \in [8]} \{T_i \text{ is succ.}\} \cup \bar{E} | r_{\text{Com}} \text{ is good}\right) \\ &\geq (1-a)\tilde{\epsilon} - \Pr(\bar{E} | r_{\text{Com}} \text{ is good}) \\ &\geq (1-a)\tilde{\epsilon} - \Pr(\bar{E}) \\ &\geq (1-a)\tilde{\epsilon} - \frac{12}{|\mathbb{S}|} \\ &\geq (1-a)\tilde{\epsilon} - 6\epsilon > 0 \end{aligned}$$

Let define $p_0 := (1-a)\tilde{\epsilon} - 6\epsilon$. By running $\tilde{P}$ η times with the same r_{Com} as for the successful transcript T_1 , we obtain seven successful and extractable transcripts with probability at least $\frac{1}{2}$ when

$$\eta \approx \frac{\ln(2)}{\ln(\frac{1}{1-p_0})} \leq \frac{\ln(2)}{p_0}.$$

□

C.3 Proof of Theorem 2

Proof. Completeness. Let us assume that P knows a witness (x, y, ω) for the word (R, C_1, C_2, C_3) in the language defined in Theorem 2, *i.e.*, $\text{Com}_p(x) = C_1$, $\text{Com}_p(y) = C_2$, $\text{Com}_q(\omega) = C_3$, and $\omega R = (x, y)$. If the computation described in ComDLog is honestly performed, then all the checks of V pass. The completeness is hence perfect.

Honest-verifier zero-knowledge. We build a PPT simulator $\mathcal{S}$ that outputs transcripts indistinguishable from real transcripts without knowing a valid witness. $\mathcal{S}$ takes as input (R, C_1, C_2, C_3) , and works as follows:

1. Sample at random $i^* \xleftarrow{\$} [N]$
2. Sample a PRG tree and compute parties seeds $\text{seed}_1, \dots, \text{seed}_N$
3. For each party $i \in [N] \setminus \{i^*\}$, compute:
 - a. $[\omega]_i, [r_3]_i, r_{1,i}, r_{2,i} \xleftarrow{\$} \text{PRG}(\text{seed}_i)$
 - b. $[\omega]_i R := [S]_i = (x_i, y_i)$
 - c. $C_{1,i} := x_i P + r_{1,i} Q$, $C_{2,i} := y_i P + r_{2,i} Q$, $C_{3,i} := [\omega]_i P + [r_3]_i Q$
4. Sample at random $\Delta\omega, \Delta r_3 \xleftarrow{\$} \mathbb{F}_q$
5. Compute $C_{1,i^*} := C_1 - \sum_{i \neq i^*} C_{1,i}$, $C_{2,i^*} := C_2 - \sum_{i \neq i^*} C_{2,i}$, $C_{3,i^*} := C_3 - \sum_{i \neq i^*} C_{3,i} - \Delta\omega P - \Delta r_3 Q$

6. Set $h = \mathcal{H}(\Delta\omega, \Delta r_3, \text{Com}(\text{seed}_i)_{i \in [N]}, \{C_{\ell,i}\}_{\ell \in [3], i \in [N]})$
 7. Sample at random: $\{\gamma_1, \gamma_2, \gamma_3, \gamma_4, c\} \xleftarrow{\$} \mathbb{S}, \{a_1, a_2, a_3, a_4, b_1, b_2\} \xleftarrow{\$} \mathbb{F}_p, A_{4,2} \xleftarrow{\$} E'(\mathbb{F})$
 8. Compute the commitments $A_1 = \text{Com}_p(a_1; b_1) - cX_1, A_2 = \text{Com}_p(a_2; b_2) - cX_2, A_3 = \text{Com}_p(0; a_3) - cW_3, A_{4,1} = a_1B_1 + \gamma_4a_2B_2 - A_{4,2}\gamma_4 - (C_1 + C_2\gamma_4)c - \text{Com}_p(0; a_4)$.
 9. Output the transcript
- $$(h, i^*, C_{1,i^*}, C_{2,i^*}, C_{3,i^*}, \Delta\omega, \Delta r_3, A_1, A_2, A_3, \gamma_1, \gamma_2, \gamma_3, A_{4,1}, A_{4,2}, c, \gamma_4, a_1, a_2, a_3, a_4)$$

Hence, $\mathcal{S}$ outputs a transcript that is identically distributed to the genuine transcript without knowing the witness.

Knowledge soundness. Let us first show how to extract a valid witness $(x, y, \omega, r_1, r_2, r_3)$ from nine transcripts satisfying some specific conditions. We will then show how to get such transcripts from a rewindable black-box access to $\tilde{\mathbf{P}}$, which denotes a malicious prover.

Transcripts used for extraction. Let us assume that we can extract 9 transcripts $\{T_i\}_{i \in [9]}$ from $\tilde{\mathbf{P}}$, denoting as

$$T_i := (\text{Com}_{1,T_i}, \text{Chall}_{1,T_i}, \text{Com}_{2,T_i} \wedge \text{Resp}_{1,T_i}, \text{Chall}_{2,T_i}, \text{Resp}_{2,T_i}) \text{ for } i \in [9],$$

where $\text{Chall}_{1,T_i} = (i_{T_i}^*, \gamma_{1,T_i}, \gamma_{2,T_i}, \gamma_{3,T_i})$, and such that $\text{Com}_{1,T_1} = \text{Com}_{1,T_i}$ for $i \in [9]$. Moreover, we add the following constraints:

- $T_1, \dots, T_8$ are satisfying the same conditions as the eight *extractable* transcripts from the soundness security proof of Theorem 1 (see proof C.2),
- T_9 has a distinct challenge $i_{T_9}^* \in \text{Chall}_{1,T_9}$ from at least another transcript, let us say T_1 , i.e. $i_{T_9}^* \neq i_{T_1}^*$.

Suppose that these nine transcripts are successful. Then, from T_1 and T_9 we can recover the witness ω and its randomness r_3 since we get all the shares of the sharings $[\![\omega]\!]$, $[\![r_3]\!]$ committed in $\text{Com}_{1,T_1} = \text{Com}_{1,T_9}$. To recover the randomness r_1, r_2 , we can use the same strategy as the one used in the proof of soundness for **AddPoint** (see proof C.2).

Extraction from $\tilde{\mathbf{P}}$. Now, let us analyze how we can get such transcripts.

Let K be the set of randomness in **ComDLog** and let $\text{Succ}_{\tilde{\mathbf{P}}} \subset K^9$ be the set of randomness from which one can generate nine different successful transcripts of **ComDLog**. By hypothesis, we have $\Pr[(k_1, \dots, k_9) \in \text{Succ}_{\tilde{\mathbf{P}}}] = \tilde{\epsilon}$. We shall further denote by r_{Com} the randomness of $\tilde{\mathbf{P}}$ which is used to generate the initial commitment Com_{1,T_1} . Let us fix an arbitrary value $\alpha \in (0, 1)$ such that $(1 - \alpha)\tilde{\epsilon} > 6\epsilon$ (assuming that $\tilde{\epsilon} > 6\epsilon$). Let κ be a possible realization of K . We say that κ is good if it is such that

$$\Pr[(\kappa, k_2, \dots, k_9) \in \text{Succ}_{\tilde{\mathbf{P}}} | \kappa \in K] \geq (1 - \alpha)\tilde{\epsilon}.$$

Then, by the Splitting Lemma 1 we have

$$\Pr[\kappa \text{ is good} | \text{Succ}_{\tilde{\mathbf{P}}}] \geq \alpha.$$

To obtain all the transcripts we run the protocol a first time to get T_1 . Using the randomness of T_1 , we can obtain further successful transcripts with high probability by rewinding the protocol after the initial commitment. One formally describes the extractor routine below, where η is a parameter that will be specified further in the proof. A set of eight transcripts is said to be *extractable* if it satisfies the previously stated conditions required for transcripts to be used in extraction.

For this, we use one extractor that firstly extracts two transcripts to recover the witnesses for the MPC part and then another extractor to extract eight other transcripts using the same r_{Com} to obtain the witnesses for **AddPoint**.

Extractor **E** (on input a successful transcript T_1):

1. Do N_1 times:
2. Run $\tilde{P}$ with honest V and same r_{Com} as T_0 to get eight transcript
3. If the eight transcripts are successful and *extractable* transcripts (see proof C.2)
4. return $T_2, \dots, T_8$

Extractor **E_{ComDLog}**:

1. Repeat $+\infty$ times
2. Run $\tilde{P}$ with honest V to get T_0 , go to next iteration if T_0 is unsuccessful
3. Call $\mathcal{E}$ on T_0 to get $T_1, \dots, T_7$
4. Do N times
5. Run $\tilde{P}$ with same r_{Com} as T_0 to get one more transcripts T_8
6. If T_8 is a successful transcripts s.t $i_{T_8}^* \neq i_{T_0}^*$
7. return $T_0, \dots, T_8$

Let us evaluate the probability that **E_{ComDLog}** outputs such transcripts. Consider a loop iteration during which nine successful transcripts have been found. We analyze the probability that these transcripts are also extractable. Moreover, we know from proof C.2 that there is an extractor for **Addpoint** that extracts a witness with probability $\frac{2}{|\mathbb{S}|}$

$$\begin{aligned}
& \Pr\left(\bigcap_{i \in [9]} \{T_i \text{ is successful}\} \cap E_1 \cap E_2 \mid r_{\text{Com}} \text{ is good}\right) \\
&= \Pr\left(\bigcap_{i \in [8]} \{T_i \text{ is succ.}\} \mid r_{\text{Com}} \text{ is good}\right) - \Pr\left(\bigcap_{i \in [8]} \{T_i \text{ is succ.}\} \cup \bar{E}_1 \cup \bar{E}_1 \mid r_{\text{Com}} \text{ is good}\right) \\
&\geq (1-a)\tilde{\epsilon} - \Pr(\bar{E}_1 \mid r_{\text{Com}} \text{ is good}) - \Pr(\bar{E}_2 \mid r_{\text{Com}} \text{ is good}) \\
&\geq (1-a)\tilde{\epsilon} - \Pr(\bar{E}_1) - \Pr(\bar{E}_2) \\
&\geq (1-a)\tilde{\epsilon} - 2 \max\left(\frac{1}{N}, \frac{2}{|\mathbb{S}|}\right) \\
&\geq (1-a)\tilde{\epsilon} - 2\epsilon > 0
\end{aligned}$$

Let define $p_0 := (1-a)\tilde{\epsilon} - 2\epsilon$. By running $\tilde{P}$ η times with the same r_{Com} as for the successful transcript T_1 , we obtain seven successful and extractable transcripts with probability at least $\frac{1}{2}$ when

$$\eta \approx \frac{\ln(2)}{\ln(\frac{1}{1-p_0})} \leq \frac{\ln(2)}{p_0}.$$

□

D Additional Protocols

D.1 UserSetup for Protocol 4

Protocol 4 (Section 4) invokes **UserSetup** on input $\text{pp} := (\text{pp}_1, \text{pp}_2, \dots)$. One describes this routine here.

► **UserSetup(pp)**
 $(\text{sk}_{\text{LHE}_q}, \text{pk}_{\text{LHE}_q}) \leftarrow \text{LHE}_q.\text{KeyGen}(\text{pp}_1)$
 $(\text{sk}_{\text{LHE}_p}, \text{pk}_{\text{LHE}_p}) \leftarrow \text{LHE}_p.\text{KeyGen}(\text{pp}_2)$
Output $(\text{sk}_{\text{LHE}_q}, \text{pk}_{\text{LHE}_q}, \text{sk}_{\text{LHE}_p}, \text{pk}_{\text{LHE}_p})$

D.2 Protocol NComDLog from Section 5

Protocol 5 - NIZK argument of a committed discrete logarithm

Public input: $R \in E(\mathbb{F}_p)$, $C_1 = \text{Com}_p(x; r_1)$, $C_2 = \text{Com}_p(y; r_2)$, $C_3 = \text{Com}_q(\omega; r_3)$

Prover P input: $\{r_1, r_2\} \in \mathbb{F}_p$, $\{r_3, \omega\} \in \mathbb{F}_q$

Step 1 P produces the MPCitH part. For each iteration $t \in [\tau]$:

1. Compute a PRG tree with N leaves $\{\text{seed}_i^{(t)}\}_{i \in [N]}$, derive randomness, and compute each party's shares $\{\llbracket r_3 \rrbracket_i^{(t)}, \llbracket \omega \rrbracket_i^{(t)}\}$ (with auxiliary value $\{\Delta r_3^{(t)} = r_3 - \sum_{i=1}^N \llbracket r_3 \rrbracket_i^{(t)}, \Delta \omega^{(t)} = \omega - \sum_{i=1}^N \llbracket \omega \rrbracket_i^{(t)}\}$).
2. Derive $\llbracket S \rrbracket_i^{(t)} = \llbracket \omega \rrbracket_i^{(t)} \cdot R$, $\Delta S^{(t)} = \Delta \omega^{(t)} \cdot R$, and $(x_i^{(t)}, y_i^{(t)}) = \llbracket S \rrbracket_i^{(t)}$ for every $i \in [N]$.
3. Compute commitments $C_{1,i}^{(t)} = \text{Com}_p(x_i^{(t)}; r_{1,i}^{(t)})$, $C_{2,i}^{(t)} = \text{Com}_p(y_i^{(t)}; r_{2,i}^{(t)})$ and $C_{3,i}^{(t)} = \text{Com}_q(\llbracket \omega \rrbracket_i^{(t)}; \llbracket r_3 \rrbracket_i^{(t)})$, where $r_{1,i}^{(t)}, r_{2,i}^{(t)}$ come from $\text{seed}_i^{(t)}$ for every $i \in [N]$.
4. Compute $h^{(t)} = \mathcal{H}(\Delta \omega^{(t)}, \Delta r_3^{(t)}, \text{Com}(\text{seed}_i^{(t)})_{i \in [N]}, \{C_{\ell,i}^{(t)}\}_{\ell \in [3], i \in [N]})$.
5. Compute the challenge for opening views $\mathcal{H}(C_1, C_2, C_3, h^{(t)}) := j^{(t)} \in [N]$.
6. Open the views to all the parties except the $j^{(t)}$ -th, by computing the co-path in the PRG tree for the $\text{seed}_{j^{(t)}}^{(t)}$ and sending $\Delta r_3^{(t)}, \Delta \omega^{(t)}, \text{Com}(\text{seed}_{j^{(t)}}^{(t)}), \{C_{\ell,j^{(t)}}^{(t)}\}_{\ell \in [3]}$.

Step 2 CDLSS part: P first computes $(x, y) = \omega \cdot R$ and then defines:

$$\begin{aligned} A &= (x, y), \\ B^{(t)} &= -\llbracket S \rrbracket_{j^{(t)}}^{(t)}, \\ C^{(t)} &= \sum_{j \neq j^{(t)}} \llbracket S \rrbracket_j^{(t)} + \Delta \omega^{(t)} \cdot R. \end{aligned}$$

P uses $(A, B^{(t)}, C^{(t)})_{t \in [\tau]}$ as input to compute τ CDLSS's first message in order to produce a Fischlin transformation of CDLSS as described in Section 5. P gets π_{CDLSS} .

Step 3 P outputs the complete proof

$$\pi = ((h^{(t)}, \Delta r_3^{(t)}, \Delta \omega^{(t)}, \text{Com}(\text{seed}_{j^{(t)}}^{(t)}), \{C_{\ell,j^{(t)}}^{(t)}\}_{\ell \in [3]}, \text{co-path}_{\text{seed}_{j^{(t)}}^{(t)}})_{t \in [\tau]}, \pi_{\text{CDLSS}}).$$

E Blind ECDSA Instantiation

Section 7 presented an instantiation of Protocol 4. We now describe the complete protocol as instantiated here.

Routines. The following routines use the notations introduced in Section 7, in particular: $E'(\mathbb{F})$ is a group of prime order p , $E(\mathbb{F}_p)$ of prime order q , and other algorithms are defined in Appendix A.

► $\text{SetupEG}(E'(\mathbb{F}), p)$
 $G_{\text{EG}} \leftarrow$ a generator of $E'(\mathbb{F})$
 $(\text{pk}_{\text{EG}}, \text{sk}_{\text{EG}}) \leftarrow \text{EG.KeyGen}(E'(\mathbb{F}), p, G_{\text{EG}})$
 Output $(\text{pk}_{\text{EG}}, \text{sk}_{\text{EG}})$

► $\text{SetupPC}(E(\mathbb{F}_p), q)$
 $P, Q \leftarrow$ generators of $E(\mathbb{F}_p)$
 Output (P, Q)

► $\text{SetupCL}(1^\lambda, E(\mathbb{F}_p), q)$
 $(\text{pk}_{\text{CL}}, \text{sk}_{\text{CL}}) \leftarrow \text{CL.KeyGen}(1^\lambda, q)$, where in $\text{CL.Gen}(1^\lambda, q)$ the subgroup F of order q (according to the notation from Appendix A) is set as $E(\mathbb{F}_p)$.
 Output $(\text{pk}_{\text{CL}}, \text{sk}_{\text{CL}})$

► $\text{UserSetup}(1^\lambda, E'(\mathbb{F}), E(\mathbb{F}_p), p, q)$
 $(\text{pk}_{\text{EG}}, \text{sk}_{\text{EG}}) \leftarrow \text{SetupEG}(E'(\mathbb{F}), p)$
 $(\text{pk}_{\text{CL}}, \text{sk}_{\text{CL}}) \leftarrow \text{SetupCL}(1^\lambda, E(\mathbb{F}_p), q)$
 $(P, Q) \leftarrow \text{SetupPC}(E(\mathbb{F}_p), q)$
 $(P', Q') \leftarrow \text{SetupPC}(E'(\mathbb{F}), p)$
 Output $(\text{pk}_{\text{EG}}, \text{sk}_{\text{EG}}, \text{pk}_{\text{CL}}, \text{sk}_{\text{CL}}, P, Q, P', Q')$

Instantiation.

Protocol 6 - Instantiation of the blind ECDSA (Protocol 4)**► BS.Setup(1^λ)**

Generate $E(\mathbb{F}_p)$ of order q , $E'(\mathbb{F})$ of order p , a generator G of $E'(\mathbb{F})$

Generate a SHA-256 cryptographic hash function $\mathcal{H}$

$\text{pp} \leftarrow (1^\lambda, E(\mathbb{F}_q), E'(\mathbb{F}), q, p, G, \mathcal{H})$

Output pp

► BS.KeyGen(pp)

$\text{sk}_{\text{ECDSA}} \xleftarrow{\$} \mathbb{F}_q$

$\text{pk}_{\text{ECDSA}} \leftarrow \text{sk}_{\text{ECDSA}} \cdot G$

Output $(\text{sk}_{\text{ECDSA}}, \text{pk}_{\text{ECDSA}})$

► BS.User₀($\text{pp}, m, \text{pk}_{\text{ECDSA}}$)

$(\text{sk}_{\text{EG}}, \text{pk}_{\text{EG}}, \text{sk}_{\text{CL}}, \text{pk}_{\text{CL}}, P, Q, P', Q') \leftarrow \text{UserSetup}(1^\lambda, E'(\mathbb{F}), E(\mathbb{F}), p, q)$

$k_u \xleftarrow{\$} \mathbb{F}_q$

$(x_{Q_u}, y_{Q_u}) := Q_u \leftarrow k_u G$

$E_u \leftarrow \text{EG.Enc}(Q_u; r_{E_u})$

$E_m \leftarrow \text{CL.Enc}(\mathcal{H}(m); r_{E_m})$

$C_1 \leftarrow \text{PC.Com}(x_{Q_u}; r_1)$

$C_2 \leftarrow \text{PC.Com}(y_{Q_u}; r_2)$

$C_3 \leftarrow \text{PC.Com}(k_u; r_3)$

$\pi_1 \leftarrow \text{NIZK}_{\mathcal{L}_1} \cdot \text{P}((C_1, C_2, C_3, E_u, G); (Q_u, k_u, r_1, r_2, r_3, r_{E_u}))$

$\pi_2 \leftarrow \text{NIZK}_{\mathcal{L}_2} \cdot \text{P}(E_m; (m, r_{E_m}))$

$\text{pkeys} \leftarrow (\text{pk}_{\text{ECDSA}}, \text{pk}_{\text{EG}}, \text{pk}_{\text{CL}})$

$\text{keys} \leftarrow (\text{pkeys}, \text{sk}_{\text{EG}}, \text{sk}_{\text{CL}})$

$\text{msg}_{U,0} \leftarrow (\{C_i\}_{i \in [3]}, E_u, E_m, \pi_1, \pi_2, \text{pkeys})$

$\text{st}_{U,0} \leftarrow (k_u, r_3, C_3, \text{keys})$

Output $(\text{msg}_{U,0}, \text{st}_{U,0})$

► BS.Signer₁($\text{msg}_{U,0}, \text{sk}_{\text{ECDSA}}$)

Abort if $\text{NIZK}_{\mathcal{L}_1} \cdot \text{V}(\pi_1)$ or $\text{NIZK}_{\mathcal{L}_2} \cdot \text{V}(\pi_2)$ fails

$k_s \xleftarrow{\$} \mathbb{F}_q$

$Q_s \leftarrow k_s G$

$E_Q \leftarrow k_s E_u$

$\text{msg}_{S,1} \leftarrow E_Q$

$\text{st}_S \leftarrow (E_m, k_s, Q_s, C_3, \text{sk}_{\text{ECDSA}}, \text{pkeys})$

Output $(\text{msg}_{S,1}, \text{st}_S)$

► BS.User₁($\text{msg}_{S,1}, \text{st}_{U,0}$)

$(x_Q, y_Q) = Q \leftarrow \text{CL.Dec}(E_Q)$

$Q_s \leftarrow k_u^{-1} Q$

$C_{x_Q} \leftarrow \text{PC.Com}(x_Q; r_{C_1})$

$E_{x_Q} \leftarrow \text{CL.Enc}(x_Q; r_{E_{x_Q}})$

$C_{y_Q} \leftarrow \text{PC.Com}(y_Q; r_{C_2})$

$\pi_3 \leftarrow \text{NIZK}_{\mathcal{L}_3} \cdot \text{P}((C_{x_Q}, C_{y_Q}, C_3, E_{x_Q}, Q_s); (k_u, Q, r_{C_1}, r_{E_{x_Q}}, r_{C_2}, r_3))$

$\text{msg}_{U,1} \leftarrow (C_{x_Q}, C_{y_Q}, E_{x_Q}, \pi_3)$

$\text{st}_{U,1} \leftarrow (x_Q, k_u, \text{keys})$

Output $(\text{msg}_{U,1}, \text{st}_{U,1})$

► BS.Signer₂($\text{msg}_{U,1}, \text{st}_S$)

If $\text{NIZK}_{\mathcal{L}_3} \cdot \text{V}(\pi_3)$ fails, abort.

$\tilde{s} \leftarrow k_s^{-1}(E_m + \text{sk}_{\text{ECDSA}} E_{x_Q})$

$\pi_4 \leftarrow \text{NIZK}_{\mathcal{L}_4}(Q_s, \text{pk}_{\text{ECDSA}}, E_m, E_{x_Q}, \tilde{s};$

$k_s^{-1}, \text{sk}_{\text{ECDSA}})$

$\text{msg}_{S,2} \leftarrow (\tilde{s}, \pi_4)$

Output $\text{msg}_{S,2}$

▶ $\text{BS.User}_2(\text{st}_{U,1}, \text{msg}_{S,2})$
 If $\text{NIZK}_{\mathcal{L}_4}.\text{V}(\pi_4)$ fails, abort.
 $s \leftarrow k_u^{-1} \text{CL.Dec}(\tilde{s})$
 $\sigma \leftarrow (x_Q, s)$
 Output σ

▶ $\text{Verify}(m, \sigma, \text{pk})$
 $(x, s) \leftarrow \sigma$
 $(x', y') \leftarrow H(m)s^{-1}(G + x \cdot \text{pk}_{\text{ECDSA}})$
 Output $(x = x')$